



# Reciprocating Gas Compressor Hardware-in-the-Loop Simulator

Environment Setup and Model Architecture Reference

Nominally Based on an Ariel JGH/4-Class Compressor and a CAT G3516LE-Class Engine  
Simulink Plant Model / CODESYS Soft-PLC Controller

Prepared by: Anas Alanqar

Prepared for: Maikana Automation

August 6, 2026

This document has three parts: Part I introduces the system and the physics it models; Part II is an architecture reference for the Simulink plant model and the CODESYS sequencer; Part III is a reproducible environment setup procedure.

# Contents

|           |                                                          |           |
|-----------|----------------------------------------------------------|-----------|
| <b>I</b>  | <b>Introduction and Background</b>                       | <b>6</b>  |
| 0.1       | Purpose and Scope . . . . .                              | 7         |
| 0.2       | Reciprocating Compressor Background . . . . .            | 7         |
| 0.2.1     | Pressure as a Consequence of Mass Accumulation . . . . . | 8         |
| 0.3       | Governing Equations . . . . .                            | 9         |
| 0.3.1     | What Is Deliberately Absent . . . . .                    | 16        |
| <br>      |                                                          |           |
| <b>II</b> | <b>Model Architecture</b>                                | <b>18</b> |
| 0.1       | Top-Level Topology . . . . .                             | 19        |
| 0.2       | Valve Actuator Pattern . . . . .                         | 21        |
| 0.3       | Bypass . . . . .                                         | 24        |
| 0.4       | Cooling . . . . .                                        | 25        |
| 0.5       | Lubrication . . . . .                                    | 28        |
| 0.6       | Instrumentation . . . . .                                | 30        |
| 0.7       | Fault Injection . . . . .                                | 34        |
| 0.8       | Logging vs. the True OPC UA Boundary . . . . .           | 38        |
| 0.9       | OPC UA Interface Mechanics . . . . .                     | 41        |
| 0.9.1     | Variant Source Rationale . . . . .                       | 43        |
| 0.10      | PLC Sequencer . . . . .                                  | 43        |
| 0.10.1    | State Diagram . . . . .                                  | 44        |
| 0.10.2    | Step Table . . . . .                                     | 46        |
| 0.10.3    | Constants — Current Test Values . . . . .                | 47        |
| 0.10.4    | USD vs. Normal Stop . . . . .                            | 50        |
| 0.10.5    | Verification . . . . .                                   | 50        |

|            |                                                            |           |
|------------|------------------------------------------------------------|-----------|
| <b>III</b> | <b>Environment Setup Guide</b>                             | <b>52</b> |
| 0.1        | CODESYS IDE and Control Win Runtime Installation . . . . . | 53        |
| 0.2        | Npcap Installation . . . . .                               | 53        |
| 0.3        | Project Creation . . . . .                                 | 53        |
| 0.4        | Symbol Publishing Configuration . . . . .                  | 54        |
| 0.5        | Device Security Settings . . . . .                         | 54        |
| 0.6        | Runtime Security Policy . . . . .                          | 56        |
| 0.7        | First-Login Credentials . . . . .                          | 58        |
| 0.8        | Simulink OPC UA Client Configuration . . . . .             | 58        |
| 0.9        | Re-Browsing OPC UA Nodes After a CODESYS Rebuild . . . . . | 59        |
| 0.10       | Known Gotchas . . . . .                                    | 59        |

# List of Figures

|    |                                                                                                                                                                                                                                                                                                                                                                 |    |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Process flow of the three-stage reciprocating compressor package, showing the main gas path, the bypass recycle loop, and the blowdown vent. Stage pressures correspond to the model design point. . . . .                                                                                                                                                      | 8  |
| 2  | Stage pressure distribution at the converged design point; the three stage ratios ( $\approx 2.97$ each) confirm equal-ratio staging per eq. (2). . . . .                                                                                                                                                                                                       | 11 |
| 3  | Simulated startup pressure profile across all four stage pressures, demonstrating the mass-accumulation dynamics of eq. (8). . . . .                                                                                                                                                                                                                            | 13 |
| 4  | Top-level topology of <code>compressor_slice.slx</code> : the full subsystem canvas, orientation only. . . . .                                                                                                                                                                                                                                                  | 20 |
| 5  | Blowdown valve actuator internals: <code>gain(100)</code> $\rightarrow$ error junction $\rightarrow$ <code>gain(20)</code> $\rightarrow$ rate limiter $\rightarrow$ integrator. . . . .                                                                                                                                                                         | 21 |
| 6  | Blowdown subsystem interface context, showing the OPC UA manual-override mux on the command input and the open/closed limit-switch pair on the position feedback. . . . .                                                                                                                                                                                       | 21 |
| 7  | Suction ESD actuator internals, matching the pattern of Figure 5. . . . .                                                                                                                                                                                                                                                                                       | 22 |
| 8  | Suction ESD interface context, matching the pattern of Figure 6. . . . .                                                                                                                                                                                                                                                                                        | 22 |
| 9  | Discharge ESD actuator internals, matching the pattern of Figure 5. . . . .                                                                                                                                                                                                                                                                                     | 23 |
| 10 | Discharge ESD interface context, matching the pattern of Figure 6. . . . .                                                                                                                                                                                                                                                                                      | 23 |
| 11 | Valve position transients showing actuator sequencing order and blowdown's fail-open initial condition (100% at $t = 0$ ). Measured slew rates for the 5-95% traverse (4.5 s for both ESD valves at 20%/s, 1.8 s for blowdown at 50%/s) are reported by the verification suite; the rate difference is not visually distinguishable at this time scale. . . . . | 23 |
| 12 | Bypass actuator internals: <code>gain</code> $\rightarrow$ rate limiter $\rightarrow$ integrator core, no <code>gain(100)</code> front end. . . . .                                                                                                                                                                                                             | 24 |
| 13 | Bypass interface context, showing <code>bypass_rate_faulted</code> in and <code>bypass_rate_raw</code> out; no override mux, no limit switches. . . . .                                                                                                                                                                                                         | 25 |
| 14 | Cooling subsystem interface: cooler command inputs, fan-count output, and the resulting temperature outputs. . . . .                                                                                                                                                                                                                                            | 26 |
| 15 | Cooling subsystem internals: fan-count derivation, lookup tables, and first-order lag states. . . . .                                                                                                                                                                                                                                                           | 27 |

|    |                                                                                                                                                                                                                                                                                                                          |    |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 16 | Aftercooler and intercooler outlet temperature response for 0/1/2 fans running (eq. (16)). Note the intercooler trace is a direct lookup with no first-order lag applied (unlike the aftercooler), so it steps rather than curves — this is correct model behavior, not a missing dynamic. . . . .                       | 27 |
| 17 | Lubrication interface context: the shared <code>CMD_aux_lube</code> OPC UA override mux feeding both lubrication subsystems. . . . .                                                                                                                                                                                     | 28 |
| 18 | Lube_Oil_Pressure internals: running/prelube target selection into a first-order lag.                                                                                                                                                                                                                                    | 28 |
| 19 | Engine_Oil_Temp internals: operating/ambient target selection into a first-order lag.                                                                                                                                                                                                                                    | 29 |
| 20 | The transient lube oil pressure dip as the running-target formula briefly undercuts the prelube target during the crossover, per eq. (15). . . . .                                                                                                                                                                       | 29 |
| 21 | Instrumentation subsystem context: lead figure showing all instrument sub-blocks and their signal sources. . . . .                                                                                                                                                                                                       | 30 |
| 22 | Discharge_Pressure internals: three flow terms summed, scaled by $T_{ac}$ , and integrated.                                                                                                                                                                                                                              | 30 |
| 23 | Temperatures internals: two parallel first-order lag chains. . . . .                                                                                                                                                                                                                                                     | 31 |
| 24 | Cylinder discharge and cooler outlet temperatures over a full startup, showing why cylinder 2 (fed by the intercooler) is more sensitive to fan loss than cylinder 1 (eq. (11)).                                                                                                                                         | 32 |
| 25 | Kelvin to F reusable masked converter block internals. . . . .                                                                                                                                                                                                                                                           | 32 |
| 26 | P_converter (Pa to psig) reusable masked converter block internals. . . . .                                                                                                                                                                                                                                              | 33 |
| 27 | P_converter mask parameters dialog: per-instance saturation range limits. . . . .                                                                                                                                                                                                                                        | 33 |
| 28 | Fault Injection subsystem interface: the ten raw $\rightarrow$ faulted signal pairs. . . . .                                                                                                                                                                                                                             | 34 |
| 29 | Fault Injection mask parameters: the operator-facing checkbox fault-selection menu.                                                                                                                                                                                                                                      | 35 |
| 30 | Fault Injection internals: oil pressure, oil rate, and speed-reference ( <code>N_ref</code> ) faults. .                                                                                                                                                                                                                  | 36 |
| 31 | Fault Injection internals: RPM/overspeed and blocked-discharge faults. . . . .                                                                                                                                                                                                                                           | 36 |
| 32 | Fault Injection internals: cylinder temperature bias and stuck-bypass faults. . . . .                                                                                                                                                                                                                                    | 37 |
| 33 | Fault Injection internals: signal-freeze and cooler-trip faults. . . . .                                                                                                                                                                                                                                                 | 37 |
| 34 | Logging subsystem interface: the full internal telemetry set tapped for verification. .                                                                                                                                                                                                                                  | 39 |
| 35 | Logging dual-tap template: one internal signal feeding both a <code>To Workspace</code> block and, for the subset that crosses the OPC UA boundary, an <code>OPC UA Write</code> block. This pattern repeats once per tapped signal. . . . .                                                                             | 40 |
| 36 | OPC UA Read block parameters: one node per block instance. . . . .                                                                                                                                                                                                                                                       | 41 |
| 37 | OPC UA Write block parameters: one node per block instance. . . . .                                                                                                                                                                                                                                                      | 42 |
| 38 | Variant Source block parameters: activation time = update diagram, port conditions on <code>USE OPCUA</code> . . . . .                                                                                                                                                                                                   | 43 |
| 39 | PLC sequencer state diagram. The “any state” node is a diagram-notation convenience representing the USD-override behavior (Unconditional Shutdown can preempt from any running step, per the unconditional priority check executed at the top of every scan cycle) — it is <i>not</i> an actual PLC step value. . . . . | 45 |

|    |                                                                                                                                                                                                                                                                                                                          |    |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 40 | Engine speed, lube oil pressure, and engine oil temperature during startup, with the sequencer's permissive thresholds marked, showing why each permissive check must wait on a genuine physical transient. Compressor lube oil pressure settles at 119.9 psig, consistent with the model's acceptance criteria. . . . . | 48 |
| 41 | Discharge pressure following an identical stop command under normal stop and USD, demonstrating the divergent end states discussed above. . . . .                                                                                                                                                                        | 51 |
| 42 | CODESYS Devices tree for a Standard Project, showing Task Configuration and Communication Manager present. . . . .                                                                                                                                                                                                       | 54 |
| 43 | Menu location for Device Security Settings. . . . .                                                                                                                                                                                                                                                                      | 55 |
| 44 | Device Security Settings dialog with the two required non-default values: Communication Mode = ALL, Activation = ACTIVATED. . . . .                                                                                                                                                                                      | 55 |
| 45 | Menu location for the Runtime Security Policy. . . . .                                                                                                                                                                                                                                                                   | 56 |
| 46 | Runtime Security Policy dialog: Optional encryption, Code Signing = All, Optional user management, anonymous login enabled. . . . .                                                                                                                                                                                      | 57 |
| 47 | Simulink Configuration Parameters, OPC Configuration pane, showing the configured client and its connection status. . . . .                                                                                                                                                                                              | 58 |
| 48 | OPC UA namespace browse dialog used to re-select a node inside a Simulink OPC UA Read/Write block after a CODESYS rebuild. . . . .                                                                                                                                                                                       | 59 |

## Part I

# Introduction and Background

## 0.1 Purpose and Scope

This document describes a hardware-in-the-loop (HIL) simulator: a Simulink process model stands in for a physical reciprocating compressor package, exchanging signals over OPC UA with a real PLC program running on a CODESYS soft-PLC. The PLC program under test is unaware that it is talking to a simulation rather than a physical machine — from the controller’s point of view, the OPC UA boundary looks identical either way.

Simulators of this kind are useful for several general reasons. Control logic can be developed and tested before hardware exists or is available on site. Fault conditions that would be dangerous, damaging, or prohibitively expensive to create on real equipment — loss of lube oil, cooler trips, blocked discharge, sensor failures — can be injected repeatably and safely, as many times as needed, without risk to personnel or plant. Sequences that would otherwise take real thermal and pressure time constants to play out can be exercised in minutes rather than hours. And operator or engineer training can happen without committing real plant time or equipment wear. These are the general benefits of process simulators as a class of tool, not claims about this particular project’s outcomes.

What this particular model is, and is not, for should be stated plainly. The model exists to exercise and validate PLC sequencing logic: startup, shutdown, fault handling, and the state transitions between them. It is deliberately simplified in its physics — the full list of what is intentionally left out is given in Section 0.3.1 — and it is parameterized to represent a generic three-stage reciprocating compressor package, nominally based on an Ariel JGH/4-class machine driven by a CAT G3516LE-class gas engine, rather than validated against real performance data from those specific machines. It is *not* a performance prediction tool, *not* a rod-load or mechanical design tool, and *not* suitable for guaranteeing machine performance. This scoping statement is intended to protect the reader from misapplying the model outside the purpose it was built for.

This report is organized in three parts. Part I (this part) introduces the system and the physics it models: the background on reciprocating compression and the governing equations actually implemented. Part II is an architecture reference for the Simulink plant model and the CODESYS sequencer, subsystem by subsystem. Part III is a reproducible procedure for setting up the CODESYS/Simulink environment from scratch.

## 0.2 Reciprocating Compressor Background

A reciprocating compressor raises the pressure of a gas by drawing it into a cylinder through a suction valve, reducing the cylinder volume with a piston driven by a crankshaft, and expelling the compressed gas through a discharge valve. It is a positive-displacement machine: for a given cylinder geometry and running speed, the volume of gas moved per unit time is fixed by the swept volume of the piston, in contrast to a centrifugal machine whose flow varies continuously with pressure ratio along a characteristic curve. This makes reciprocating compressors the standard choice for gas gathering, gas lift, gas injection, and pipeline transport applications where high pressure ratios and stable throughput at varying discharge pressure are required.

A single compression stage is limited in the pressure ratio it can practically achieve: as the ratio across one cylinder increases, discharge temperature rises, mechanical (rod) loading increases, and volumetric efficiency falls because a larger fraction of the cylinder’s swept volume is consumed by clearance-gas re-expansion, an effect that worsens sharply as pressure ratio increases (equation (4)



of Section 0.3). Compressing this package’s full 30 psig to 1150 psig range in a single stage would therefore produce an impractically high discharge temperature and an impractically low volumetric efficiency. Multistage compression avoids this by splitting a large overall pressure ratio across several cylinders in series, with intercooling between stages to remove the heat of compression before the gas enters the next stage; splitting the ratio across three stages, as this package does, keeps each stage’s ratio near 2.97 and each stage’s discharge temperature manageable. For a fixed overall ratio  $r_{tot}$  split across  $k$  stages, distributing the ratio equally across all stages — so that each stage takes  $r_{stg} = r_{tot}^{1/k}$  — minimizes the total compression work summed across all stages, and is the standard design practice for multistage reciprocating machines; this is why  $r_{stg} = r_{tot}^{1/3}$  is used rather than an arbitrary split across the three stages. This is the basis for the stage pressure distribution implemented in the model and described in equation (2) of Section 0.3.

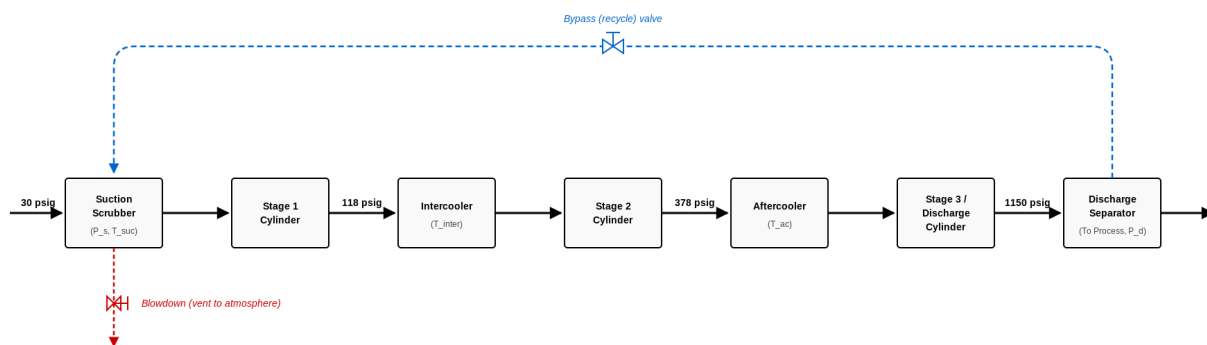


Figure 1: Process flow of the three-stage reciprocating compressor package, showing the main gas path, the bypass recycle loop, and the blowdown vent. Stage pressures correspond to the model design point.

Figure 1 shows a discharge separator as a distinct vessel, for process-schematic completeness; the model itself, however, lumps all discharge-side volume into a single discharge volume  $V_d$  (Section 0.3, equation (8)) and does not model a separator as a separate dynamic element, so the diagram should not be read as implying the model has more vessels than it does.

### 0.2.1 Pressure as a Consequence of Mass Accumulation

In this model — and in the real machine — pressure is not an independent input that drives flow. It is the opposite: pressure is the integrated *result* of mass accumulating in, or draining from, a fixed volume. A vessel does not “have” a pressure that then pushes gas around; gas flows in and out for other reasons, and whatever net mass imbalance results is what raises or lowers the pressure.

This follows directly from the ideal gas law. At fixed volume  $V$  and temperature  $T$ ,  $P = \frac{mR_{sp}T}{V}$  says pressure is directly proportional to the mass of gas contained. Differentiating with respect to time, with  $V$  and  $T$  held fixed, gives  $\frac{dP}{dt} = \frac{R_{sp}T}{V} \sum \dot{m}$  — the rate of pressure change is proportional to the *net* mass flow rate into the volume. If inflow exceeds outflow, pressure rises. If outflow exceeds inflow, pressure falls. If they balance, pressure holds steady regardless of how large either flow is. This is exactly equation (8) in Section 0.3.

This is why the two vessel pressure states ( $P_s$  for the suction volume,  $P_d$  for the discharge volume) are implemented as integrators rather than algebraic expressions: each one integrates its own net mass balance over time. Every other quantity in the model — valve positions, engine speed, cooler temperatures, compressor displacement — matters only insofar as it changes one of the mass flow terms feeding those two integrators.

Because pressure is an accumulated quantity, it responds with inertia, not instantly. Opening a vent valve does not immediately drop pressure to atmospheric; it establishes an outflow, and the pressure then decays as mass actually leaves. A large volume takes longer to depressurize than a small one at the same vent rate. This is why the blowdown and USD sequences take real time to complete rather than stepping instantly, and why the sequencer’s pressure permissive checks (P\_MAX\_START, P\_PURGE\_PERMIT) are paired with timeout timers — the logic must wait for a physical process to finish, and must fault if it does not finish in the expected time.

A concrete illustration, drawn from the model’s own structure: blowdown vents the *suction* volume, not the discharge volume. The discharge side can only lose mass through the bypass valve into suction and then out the vent. Therefore, holding blowdown open with the bypass closed drops suction pressure substantially while discharge pressure barely falls — the discharge volume has no path for mass to leave. This is a direct, observable consequence of “pressure follows mass,” and it is also the mechanism behind the USD-versus-normal-stop asymmetry documented in Section 0.10.4.

The package modeled in this report is a generic three-stage reciprocating compressor, parameterized to represent a package nominally based on an Ariel JGH/4-class machine driven by a CAT G3516LE-class gas engine. Section 0.3 discusses why this generic framing, rather than a claim of machine-specific fidelity, is the accurate one. This report models the above system as described in the following sections.

### 0.3 Governing Equations

This section states the governing equations actually implemented in the Simulink Lite model (`compressor_slice.slx`), as opposed to what a textbook treatment would ideally use. Every simplification made relative to a full first-principles model is stated explicitly, along with its justification. All pressures below are absolute Pa, all temperatures are Kelvin, and all mass flows are kg/s, with no exceptions.

#### Symbols

| Symbol     | Meaning                           | Value / units                |
|------------|-----------------------------------|------------------------------|
| $R_{sp}$   | Specific gas constant             | 345.5 J/(kg·K)               |
| $n$        | Polytropic exponent               | 1.2693                       |
| $V_{disp}$ | Swept volume per revolution       | 0.023258 m <sup>3</sup> /rev |
| $C$        | Clearance fraction                | 0.078 (calibrated)           |
| $L_{slip}$ | Slip / leakage loss fraction      | 0.04                         |
| $V_s, V_d$ | Suction, discharge vessel volumes | 3.0, 4.5 m <sup>3</sup>      |
| $N$        | Engine speed                      | rpm                          |
| $Z$        | Valve position                    | 0–100%                       |

| Symbol    | Meaning              | Value / units |
|-----------|----------------------|---------------|
| $P_{atm}$ | Atmospheric pressure | 101 325 Pa    |

### 1. Gas density — ideal gas law

$$\rho = \frac{P}{R_{sp}T} \quad (1)$$

Applied three places: suction (`rho_s`), discharge (`rho_d`, using  $T_{ac}$ ), and the supply source (`rho_src`).

**Simplification:** compressibility factor  $Z = 1$ . At 1150 psig and 245°F a real natural gas has  $Z \approx 0.85$ – $0.90$ , so densities are optimistic by roughly 10–15% at the discharge end. Acceptable because the model’s purpose is PLC logic testing, not performance guarantee — and because no gas composition analysis was supplied.

### 2. Stage pressure distribution — equal ratio

$$r_{tot} = \frac{P_d}{P_s}, \quad r_{stg} = r_{tot}^{1/3} \quad (2)$$

$$P_1 = P_s \cdot r_{stg}, \quad P_2 = P_1 \cdot r_{stg} \quad (3)$$

Three stages each take the cube root of the total ratio. This is the standard design choice for multistage reciprocating machines, per Section 0.2 — equal ratio per stage minimizes total power for a given overall ratio.

At the design point:  $r_{tot} = 26.1$ ,  $r_{stg} = 2.969$ , giving  $30 \rightarrow 118 \rightarrow 378 \rightarrow 1150$  psig.

The bar heights above match the acceptance criteria already checked by the model’s verification suite (Section 0.10.5).

**Guards in code:**  $r_{tot}$  is clamped to  $[1, 30]$ , and  $P_1$ ,  $P_2$  are clamped to not exceed  $P_d$ . Both only engage at  $t = 0$  when the package is at atmospheric and nothing is running (verified by `diagnose_clamp.m`); neither engages during normal operation.

### 3. Volumetric efficiency — clearance re-expansion

$$VE = 1 - C \left( r_{stg}^{1/n} - 1 \right) - L_{slip} \quad (4)$$

The classic reciprocating-compressor VE equation. Physically: gas trapped in the clearance volume at discharge pressure must re-expand back to suction pressure before the suction valve can open, so the cylinder never fills completely. Higher ratio  $\Rightarrow$  more re-expansion  $\Rightarrow$  less fresh gas drawn in.

At the design point:  $2.969^{1/1.2693} = 2.357$ , so  $VE = 1 - 0.078(1.357) - 0.04 = 0.854$ . Clamped to  $[0, 1]$ .

**Note:**  $C = 0.078$  was solved backwards to close the design point, because no Ariel performance data was available. This is the model’s single biggest fitted parameter — and it is precisely why this report frames the package as a generic three-stage reciprocating compressor parameterized to

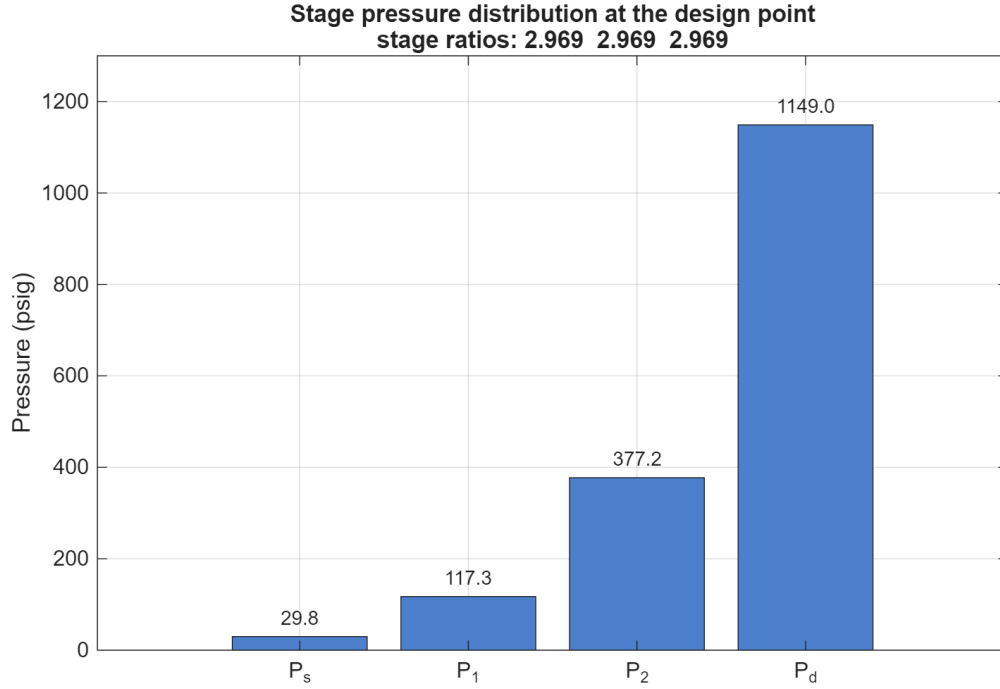


Figure 2: Stage pressure distribution at the converged design point; the three stage ratios ( $\approx 2.97$  each) confirm equal-ratio staging per eq. (2).

the class of the reference machines named in Section 0.1, not as a validated reproduction of their specific performance.

#### 4. Compressor mass flow — positive displacement

$$\dot{m}_{comp} = V_{disp} \cdot \rho_s \cdot VE \cdot \frac{N}{60} \cdot \text{gate} \quad (5)$$

$$\text{gate} = \begin{cases} 1 & \text{if } N > 200 \text{ rpm and } Z_{sesd} > 2\% \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

Swept volume per rev  $\times$  suction density  $\times$  volumetric efficiency  $\times$  revs per second. Units:  $\text{m}^3/\text{rev} \cdot \text{kg}/\text{m}^3 \cdot \text{rev}/\text{s} = \text{kg}/\text{s}$ .

The gate is what makes the compressor a *positive displacement* machine rather than a curve lookup: flow is proportional to speed and independent of pressure ratio except through  $VE$ . The  $Z_{sesd}$  term means a closed suction ESD dead-heads the cylinder — no fresh gas, therefore no flow and no compression heat.

#### 5. Valve mass flow — simplified orifice

$$\dot{m} = K \cdot \frac{Z}{100} \cdot \sqrt{\rho \cdot \max(\Delta P, 0)} \quad (7)$$

Derived from the incompressible orifice equation  $\dot{m} = C_d A \sqrt{2\rho \Delta P}$ , with  $C_d$ , the area coefficient and the factor 2 all folded into a single lumped constant  $K$  per valve. Linear valve characteristic assumed (position % maps linearly to flow area). See the flow-coefficient instances tabulated in Section 0.6 and the bypass-specific discussion in Section 0.3.

Four instances:

| Flow                          | Driving $\Delta P$ | Density      | K                      |
|-------------------------------|--------------------|--------------|------------------------|
| $\dot{m}_{sup}$ (supply in)   | $P_{src} - P_s$    | $\rho_{src}$ | $2.10 \times 10^{-3}$  |
| $\dot{m}_{byp}$ (recycle)     | $P_d - P_s$        | $\rho_d$     | $1.50 \times 10^{-3}$  |
| $\dot{m}_{proc}$ (to process) | $P_d - P_{proc}$   | $\rho_d$     | $1.335 \times 10^{-4}$ |
| $\dot{m}_{bdv}$ (vent)        | $P_s - P_{atm}$    | $\rho_s$     | $4.00 \times 10^{-3}$  |

$\dot{m}_{sup}$  is additionally gated by the suction ESD:  $(Z_{suc}/100) \cdot (Z_{sesd}/100)$  — two valves in series.

The  $\max(\Delta P, 0)$  prevents both reverse flow and a square root of a negative number. **Simplification:** no choked-flow model. At the blowdown valve venting 1150 psig to atmosphere the real flow would choke, so vent rates during a USD are approximate.

## 6. Vessel pressure dynamics — mass balance at constant volume

The physical reasoning behind this equation — that pressure is the integrated consequence of net mass flow, not an independent driver of it — is discussed at length in Section 0.2.1.

From the ideal gas law at fixed volume and temperature,  $P = \frac{m R_{sp} T}{V}$ , differentiating gives:

$$\frac{dP}{dt} = \frac{R_{sp} T}{V} \sum \dot{m} \quad (8)$$

This pair of ODEs is implemented in the `Discharge_Pressure` block described in Section 0.6.

**Suction volume:**

$$\frac{dP_s}{dt} = \frac{R_{sp} T_{suc}}{V_s} (\dot{m}_{sup} + \dot{m}_{byp} - \dot{m}_{comp} - \dot{m}_{bdv}) \quad (9)$$

with  $\frac{R_{sp}}{V_s} = \frac{345.5}{3.0} = 115.17$ .

**Discharge volume:**

$$\frac{dP_d}{dt} = \frac{R_{sp} T_{ac}}{V_d} (\dot{m}_{comp} - \dot{m}_{proc} - \dot{m}_{byp}) \quad (10)$$

with  $\frac{R_{sp}}{V_d} = \frac{345.5}{4.5} = 76.78$  — this is the `Gain(76.78)` multiplying  $T_{ac}$  into the `Product` block. (The old frozen `Constant(24299)` was this same expression evaluated at the hardcoded  $T_{ac} = 316.48$  K:  $76.78 \times 316.48 = 24\,300$ .)

These two ODEs are the core of the whole simulation. Everything else feeds them.

**Note:** blowdown vents the *suction* volume. The discharge side can only empty through the bypass into suction, then out — which is why holding blowdown open drops suction to 5 psig but discharge only to 1058 psig if the bypass is closed.

The converged values in the lower panel match the acceptance criteria already checked by the model's verification suite (Section 0.10.5).

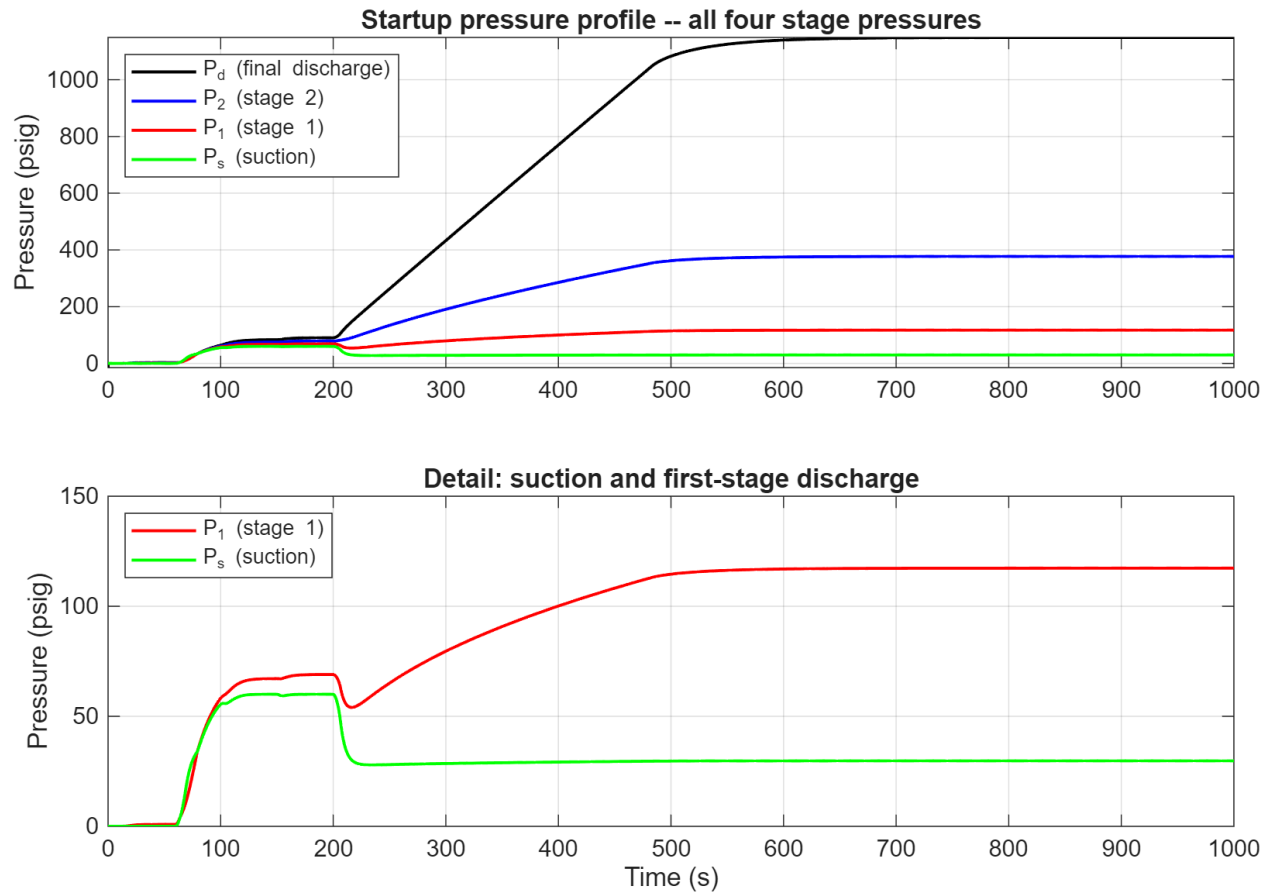


Figure 3: Simulated startup pressure profile across all four stage pressures, demonstrating the mass-accumulation dynamics of eq. (8).

## 7. Discharge temperature — polytropic compression

$$T_d = T_{in} \cdot r_{stg}^{e_T}, \quad e_T = \frac{n-1}{n} = \frac{0.2693}{1.2693} = 0.2122 \quad (11)$$

Applied per cylinder with different inlet temperatures:

| Cylinder | Inlet       | Why                               |
|----------|-------------|-----------------------------------|
| Cyl 1    | $T_{suc}$   | draws from the suction scrubber   |
| Cyl 2    | $T_{inter}$ | draws from the intercooler outlet |

$$T_d = \text{gate} \cdot (T_{in} r_{stg}^{e_T}) + (1 - \text{gate}) \cdot T_{amb} \quad (12)$$

When not running, both relax to ambient. This is why cylinder 2 is the one that trips on cooler loss (its inlet  $T_{inter}$  rises with fan count) while cylinder 1 barely moves.

**Simplification:** these are algebraic targets, not thermal states — no cylinder metal mass or thermal inertia. Discharge temperature responds instantly to ratio changes. Real cylinders lag by tens of seconds.

## 8. Valve position dynamics — rate-limited integrator

$$\frac{dZ}{dt} = \text{sat}(Z_{target} - Z, -R, +R), \quad 0 \leq Z \leq 100 \quad (13)$$

This is the actuator-lag core described from the block-diagram side in Section 0.2. The saturation on the *rate* is the stroke-time limit. Measured behaviour matches: ESD valves at  $R = 20$  %/s traverse 5→95% in 4.50 s; blowdown at  $R = 50$  %/s does 95→5% in 1.80 s.

| Valve       | R (%/s) | IC  | Target             | Failure mode        |
|-------------|---------|-----|--------------------|---------------------|
| Bypass      | —       | —   | byp_target         | —                   |
| Suction     | —       | —   | suc_target         | —                   |
| Suction ESD | 20      | 0   | 100·CMD_sesd       | fails <b>closed</b> |
| Discharge   | 20      | 0   | 100·CMD_desd       | fails <b>closed</b> |
| ESD         |         |     |                    |                     |
| Blowdown    | 50      | 100 | 100<br>(1-CMD_bdv) | · fails <b>open</b> |

The initial conditions are the safety design: on loss of instrument air or signal, the ESDs shut and the blowdown opens, venting the package. That single difference in IC is what makes a USD and a normal stop end in completely different states.

## 9. First-order lag states

$$\frac{dx}{dt} = \frac{x_{target} - x}{\tau} \quad (14)$$

Used for every quantity with real thermal or hydraulic inertia, including the lubrication pressure target of Section 0.5 and the cooling-driven temperature targets of Section 0.4:

| State                           | Target                                      | $\tau$ (s) | IC         |
|---------------------------------|---------------------------------------------|------------|------------|
| $P_{oil}$ (compressor lube)     | see eq. (15)                                | 3          | 101 325 Pa |
| $T_{oil}$ (engine oil temp)     | 361 K running, else $T_{amb}$               | 400        | 305.37 K   |
| $T_{ac}$ (aftercooler out)      | $T_{ac,target}$ if gate, else $T_{amb}$     | 60         | 305.37 K   |
| $T_{oil}$ (compressor oil temp) | $T_{oil,target}$ if running, else $T_{amb}$ | 300        | 305.37 K   |

The long  $\tau$  on  $T_{oil}$  is deliberate — it is the warm-up load permissive, crossing 100°F at roughly  $t = 200$  s. **The model computes and reports it; the PLC decides what to do with it.**

## 10. Lubrication pressure target

$$P_{oil,target} = \begin{cases} 101325 + 8.27 \times 10^5 \cdot \min\left(1, \frac{N}{850}\right) & N > 200 \\ 101325 + 3.79 \times 10^5 & \text{prelube pump commanded} \\ 101325 & \text{otherwise} \end{cases} \quad (15)$$

120 psig at rated speed, 55 psig on the prelube pump, atmospheric when off. Pressure scales linearly with speed up to 850 rpm then saturates — a pressure-relief-valve characteristic. See Section 0.5 for the block-diagram implementation.

The 101 325 offsets are what make  $P_{oil}$  a true absolute pressure like every other pressure state, so the shared Pa→psig instrument converter works on it without a special case.

**Consequence:** during start there is a real dip at  $\sim t = 160$  s. The prelube target is 55 psig, but once  $N > 200$  the running branch takes over at  $8.27 \times 10^5 \times N/850$ , which is momentarily *below* 55 psig until speed builds. A real package shows the same dip.

## 11. Cooling — fan count lookup

$$n_{fans} = \text{CMD}_{cooler1} + \text{CMD}_{cooler2} \in \{0, 1, 2\} \quad (16)$$

Three discrete lookups indexed by fan count (Multiport Switch, zero-based), implemented as described in Section 0.4:

| Target           | 0 fans           | 1 fan            | 2 fans           |
|------------------|------------------|------------------|------------------|
| $T_{inter}$      | 333.15 K (140°F) | 322.04 K (120°F) | 313.71 K (105°F) |
| $T_{ac,target}$  | 352.59 K (175°F) | 327.59 K (130°F) | 316.48 K (110°F) |
| $T_{oil,target}$ | 366.48 K (200°F) | 366.48 K (200°F) | 344.26 K (160°F) |



**Simplification:** no heat-exchanger model — no UA, no air-side flow, no approach temperature. Cooler outlet is a step function of how many fans are running. Justified because the PLC only ever sees fan *status* (on/off) and outlet *temperature*; the physics in between does not affect the logic under test.

$T_{inter}$  feeds cylinder 2's inlet (eq. (11)).  $T_{ac}$  feeds discharge density (eq. (1)) and the discharge pressure gain (eq. (8)) — which is why losing fans raises both cylinder 2's temperature and the discharge pressure.

## 12. Engine speed

Ramp-rate limited toward a commanded reference, with distinct rates by mode:

| Phase        | Rate                      |
|--------------|---------------------------|
| Cranking     | 70 rpm/s                  |
| Acceleration | 50 rpm/s                  |
| Deceleration | 75 rpm/s                  |
| Coastdown    | first-order, $\tau = 6$ s |

**Simplification:** no engine torque model, no load feedback. Speed is a commanded trajectory, not a result of torque balance against compressor load. The real Cat G3516LE with ADEM A3 governor would droop under load. Validated against an independent Python implementation to a maximum error of 1.633 rpm.

### 0.3.1 What Is Deliberately Absent

Stated plainly, because a reviewer will ask:

- **No compressibility factor** — ideal gas throughout
- **No choked flow** — all valve flows are subsonic-form
- **No thermal mass** — cylinder discharge temperatures are algebraic
- **No heat exchanger model** — cooler outlets are lookups
- **No engine torque/load coupling** — speed is commanded, not solved
- **No valve dynamics inside the cylinder** — no suction/discharge valve losses, no pulsation, no rod load
- **No sequencing logic of any kind** — by contract (see Operating Philosophy)

Every one of these is defensible for a PLC-logic test rig. None would be defensible for a performance prediction or a rod-load study, and the documentation should say so.

## Implementation — the compression MATLAB Function Block

The equations above are not merely a paper derivation: they are the actual implementation of equations 1 through 5 and 7, reproduced verbatim below from the `compression` MATLAB Function block in `compressor_slice.slx`.

```
function [m_comp, m_byp, m_proc, m_sup, m_bdv, P_1, P_2, r_stg, T_d1_t, T_d2_t, gate] = fcn(P_d,
    T_inter , T_ac, N, Z_byp, T_suc, P_s, Z_suc, P_src, Z_sesd, Z_desd, Z_bdv, data)
%#codegen
R_sp      = 345.5;
n_exp     = 1.2693;
e_T       = 0.2122;
V_disp    = 0.023258;
clearance = 0.078;
L_slip    = 0.04;
K_suc     = 2.10e-3;
K_byp     = 1.50e-3;
K_proc    = 1.335e-4;
K_bdv     = 4.00e-3;
P_proc    = 7.34082e6;
T_amb     = 305.37;
N_ct      = 200;
P_atm     = 101325;

gate = double(N > N_ct && Z_sesd > 2);

r_tot = min(max(P_d/P_s, 1), 30);
r_stg = r_tot^(1/3);
P_1 = min(P_s * r_stg, P_d);
P_2 = min(P_1 * r_stg, P_d);

VE      = min(max(1 - clearance*(r_stg^(1/n_exp) - 1) - L_slip, 0), 1);
rho_s   = P_s/(R_sp*T_suc);

m_comp = V_disp * rho_s * VE * (N/60) * gate;

rho_d = P_d/(R_sp*T_ac);           % already uses T_ac -- now it's live
rho_src = P_src/(R_sp*T_suc);
m_byp  = K_byp * (Z_byp/100) * sqrt(rho_d * max(P_d - P_s, 0));
m_sup  = K_suc * (Z_suc/100)*(Z_sesd/100) * sqrt(rho_src * max(P_src - P_s, 0));
m_proc = K_proc * (Z_desd/100)      * sqrt(rho_d * max(P_d - P_proc, 0));
m_bdv  = K_bdv * (Z_bdv/100)        * sqrt(rho_s * max(P_s - P_atm, 0));

T_d1_t = gate*(T_suc * r_stg^e_T) + (1-gate)*T_amb;
T_d2_t = gate*(T_inter * r_stg^e_T) + (1-gate)*T_amb; % already uses T_inter -- now it's live
```

$P_{proc} = 7.34082 \times 10^6$  Pa ( $\approx 1050$  psig) is confirmed from this source code and was not separately stated numerically in the governing-equations reference document.

## Part II

# Model Architecture

This part documents the Simulink plant model (`compressor_slice.slx`) and the CODESYS ST sequencer (`PLC_PRG`) that drives it over OPC UA. The Simulink model is the plant; CODESYS is the controller. Design-point numeric values throughout this part are taken from `params.m`.

## 0.1 Top-Level Topology

The model is organized as a set of subsystems representing the compressor package: valve actuators (Blowdown, Suction ESD, Discharge ESD, Bypass), Cooling, Lubrication, Instrumentation, Fault Injection, Logging, and the OPC UA interface layer connecting the whole package to the CODESYS controller. Figure 4 shows the full canvas. The compressor itself is modeled as the generic three-stage reciprocating compressor package introduced in Section 0.2; the design point is approximately 30 psig suction to 1150 psig final discharge at 0.945 kg/s throughput (see `params.m`, and the converged acceptance values in Section 0.6).

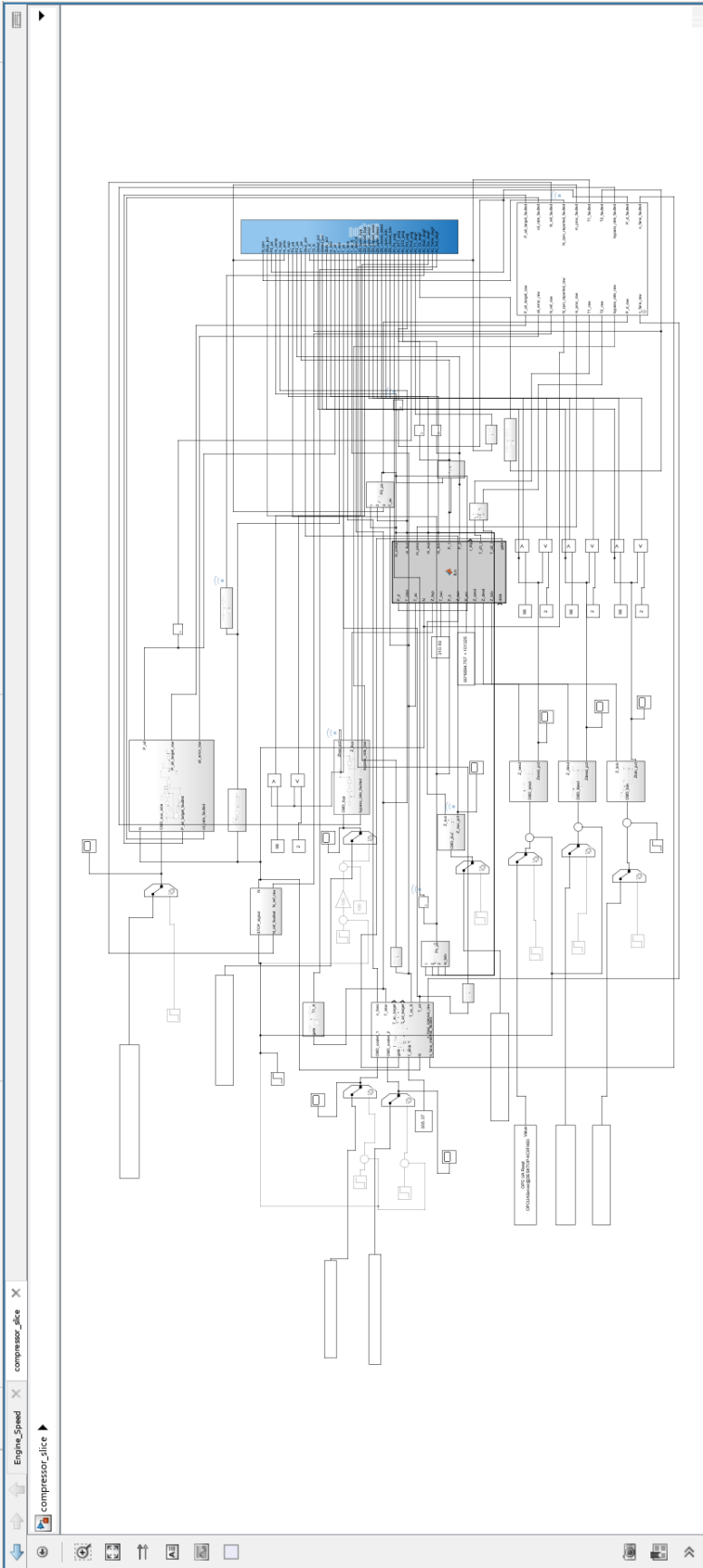


Figure 4: Top-level topology of compressor\_slice.slx: the full subsystem canvas, orientation only.

## 0.2 Valve Actuator Pattern

Blowdown, Suction ESD, and Discharge ESD are three instances of one actuator design. Blowdown is used below as the primary worked example; Suction ESD and Discharge ESD are structurally identical instances and are described by cross-reference rather than re-explained from scratch.

### Blowdown (primary example)

The actuator lag core follows this signal path: command input  $\rightarrow$  gain of 100  $\rightarrow$  error (summing) junction  $\rightarrow$  gain of 20  $\rightarrow$  rate limiter  $\rightarrow$  integrator  $\rightarrow$  position output. The position output is fed back to the error junction, forming a closed loop that tracks the commanded position through a rate-limited, integrator-based lag.

In addition to the actuator core, the Blowdown instance carries:

- An OPC UA manual-override mux on the command input, allowing the command to be sourced either from the normal sequencer path or from a manual override value written over OPC UA.
- A limit-switch pair reading the position feedback: the open limit switch asserts above 98% position, the closed limit switch asserts below 2% position.

Figure 5 shows the actuator core internals; Figure 6 shows the override mux and limit-switch pair at the subsystem boundary.

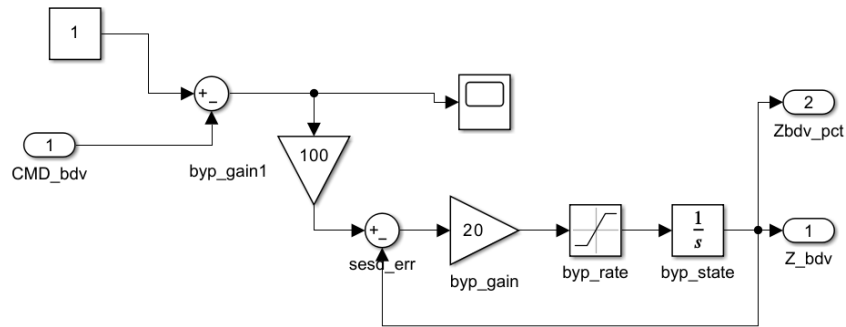


Figure 5: Blowdown valve actuator internals: gain(100)  $\rightarrow$  error junction  $\rightarrow$  gain(20)  $\rightarrow$  rate limiter  $\rightarrow$  integrator.

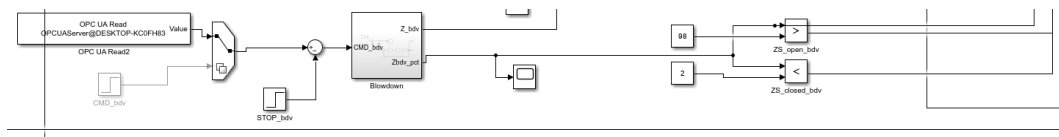


Figure 6: Blowdown subsystem interface context, showing the OPC UA manual-override mux on the command input and the open/closed limit-switch pair on the position feedback.

### Suction ESD (instance)

Suction ESD implements the identical actuator pattern described in the Blowdown example above:  $\text{gain}(100) \rightarrow \text{error junction} \rightarrow \text{gain}(20) \rightarrow \text{rate limiter} \rightarrow \text{integrator}$ , with an OPC UA manual-override mux on the command input and a 98%/2% open/closed limit-switch pair on the position feedback. See Figures 7 and 8.

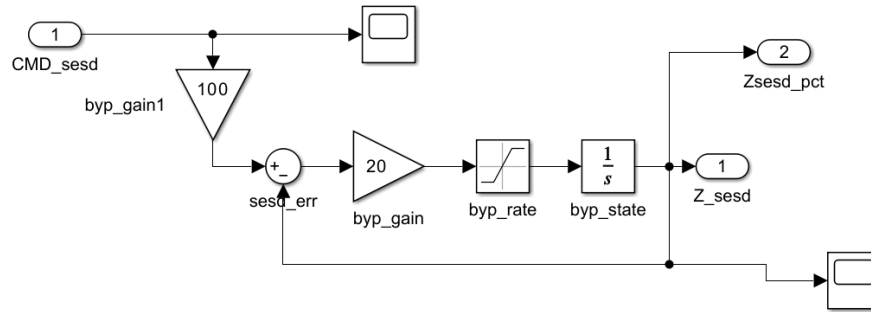


Figure 7: Suction ESD actuator internals, matching the pattern of Figure 5.

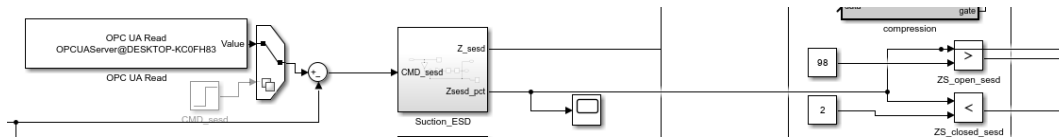


Figure 8: Suction ESD interface context, matching the pattern of Figure 6.

### Discharge ESD (instance)

Discharge ESD implements the same actuator pattern, again per Section 0.2 (Blowdown, primary example):  $\text{gain}(100) \rightarrow \text{error junction} \rightarrow \text{gain}(20) \rightarrow \text{rate limiter} \rightarrow \text{integrator}$ , OPC UA manual-override mux on the command input, 98%/2% limit-switch pair on the position feedback. See Figures 9 and 10.

These transients are consistent with the acceptance criteria already checked by the model's verification suite (Section 0.10.5).

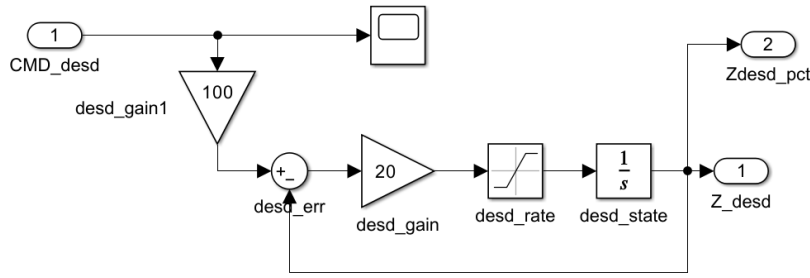


Figure 9: Discharge ESD actuator internals, matching the pattern of Figure 5.

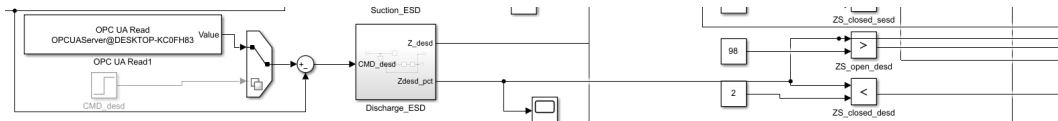


Figure 10: Discharge ESD interface context, matching the pattern of Figure 6.

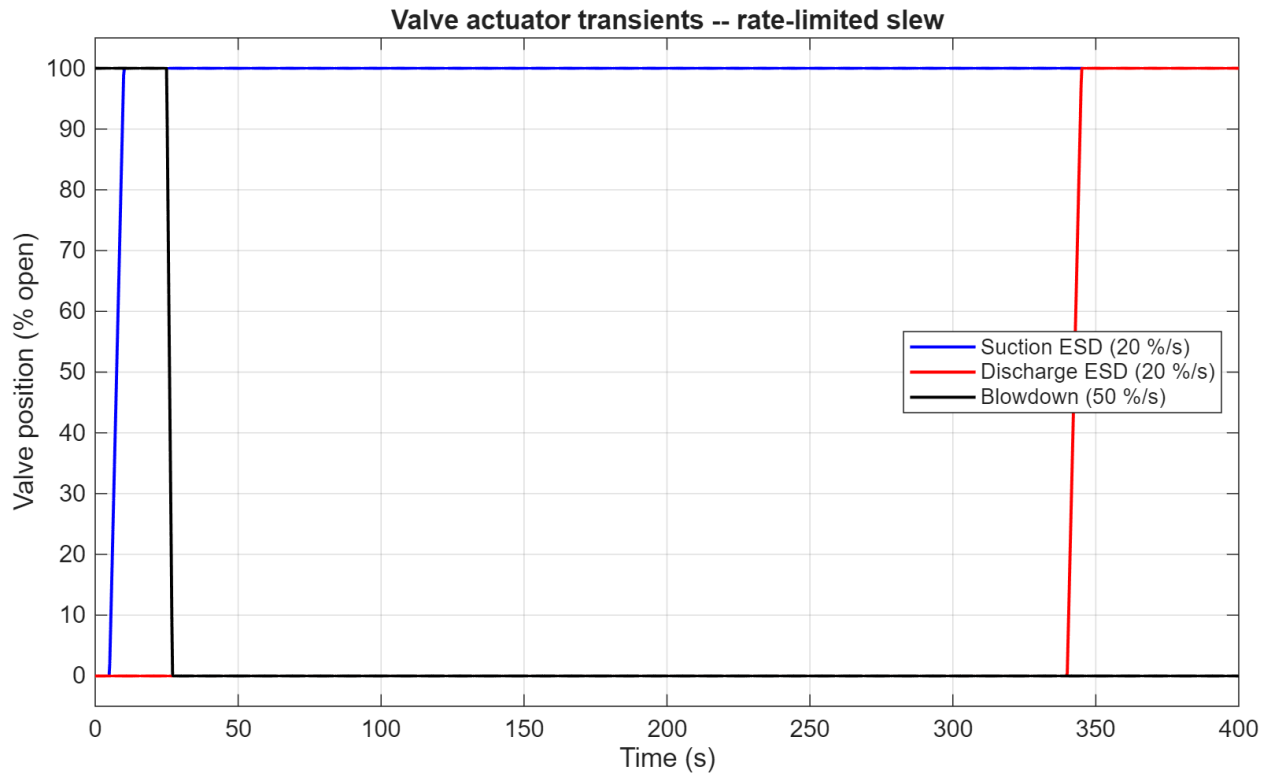


Figure 11: Valve position transients showing actuator sequencing order and blowdown's fail-open initial condition (100% at  $t = 0$ ). Measured slew rates for the 5-95% traverse (4.5 s for both ESD valves at 20%/s, 1.8 s for blowdown at 50%/s) are reported by the verification suite; the rate difference is not visually distinguishable at this time scale.



### 0.3 Bypass

The Bypass valve is related to the ESD/Blowdown family but is structurally distinct, and the distinction is deliberate rather than incidental. Bypass shares the gain  $\rightarrow$  rate-limiter  $\rightarrow$  integrator actuator-lag core used by the valve family in Section 0.2, but differs from it in two respects:

- There is no gain(100) front end ahead of the error junction.
- There is no OPC UA override mux and no limit-switch pair on the Bypass block itself.

Bypass instead exposes a fault-injection interface not present on any of the ESD valves: a `bypass_rate_faulted` input and a `bypass_rate_raw` output, allowing the Fault Injection subsystem (Section 0.7) to substitute the bypass ramp rate independently of the normal command path.

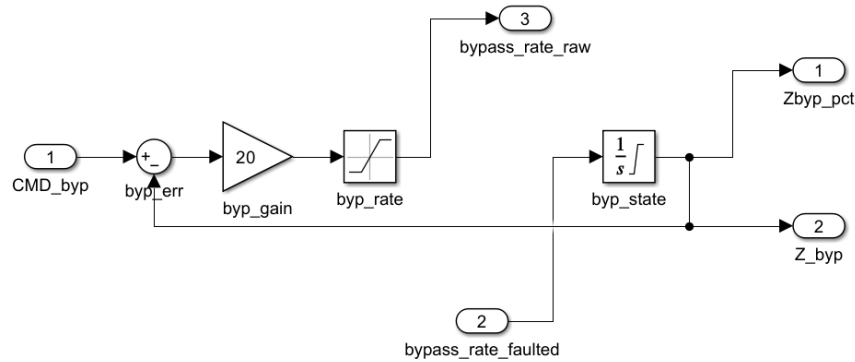


Figure 12: Bypass actuator internals: gain  $\rightarrow$  rate limiter  $\rightarrow$  integrator core, no gain(100) front end.

Bypass ramp rates from `params.m`: open at 5%/s (`rate_byp_open`), close at 15%/s (`rate_byp_close`). The bypass-to-suction flow coefficient is  $K_{byp} = 1.50 \times 10^{-3}$ .

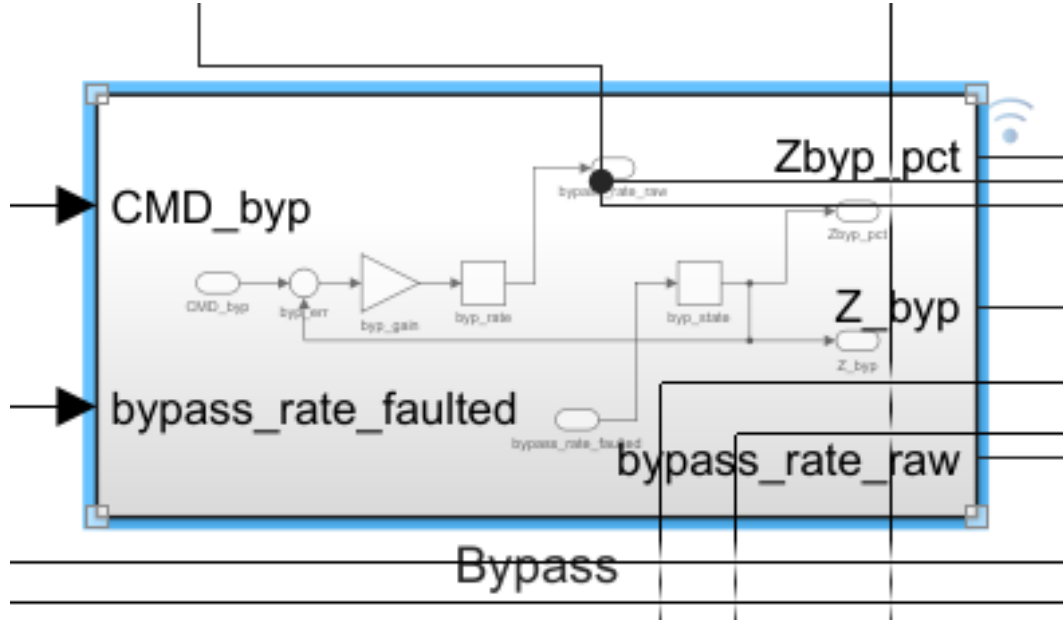


Figure 13: Bypass interface context, showing `bypass_rate_faulted` in and `bypass_rate_raw` out; no override mux, no limit switches.

## 0.4 Cooling

The Cooling subsystem derives an active fan count from the two cooler on/off commands (`CMD_cooler_1`, `CMD_cooler_2`); the fan count indexes a set of lookup tables that produce target temperatures for zero, one, or two fans running. From `params.m`:

- Intercooler target temperature table: 140°F, 120°F, 105°F for 0, 1, 2 fans.
- Aftercooler target temperature table ( $T_{ac}$ ): 175°F, 130°F, 110°F for 0, 1, 2 fans.
- Oil target temperature table ( $T_{oil}$ ): 200°F, 200°F, 160°F for 0, 1, 2 fans.

The looked-up targets drive first-order lag states for  $T_{ac}$  (time constant  $\tau_{T_{ac}} = 60$  s) and  $T_{oil}$  (time constant  $\tau_{T_{oil}} = 300$  s), so a fan-count change produces a gradual temperature response rather than a step change.

A recurring three-way switch stack, referred to in the model by the signal labels T/T1/T2, selects among a raw computed temperature and fault-substituted values. This switch stack is a reusable sub-pattern: the same three-way selection structure reappears, applied to different signals, inside the Fault Injection subsystem (Section 0.7).

These step/settle values match the acceptance criteria already checked by the model's verification suite (Section 0.10.5).

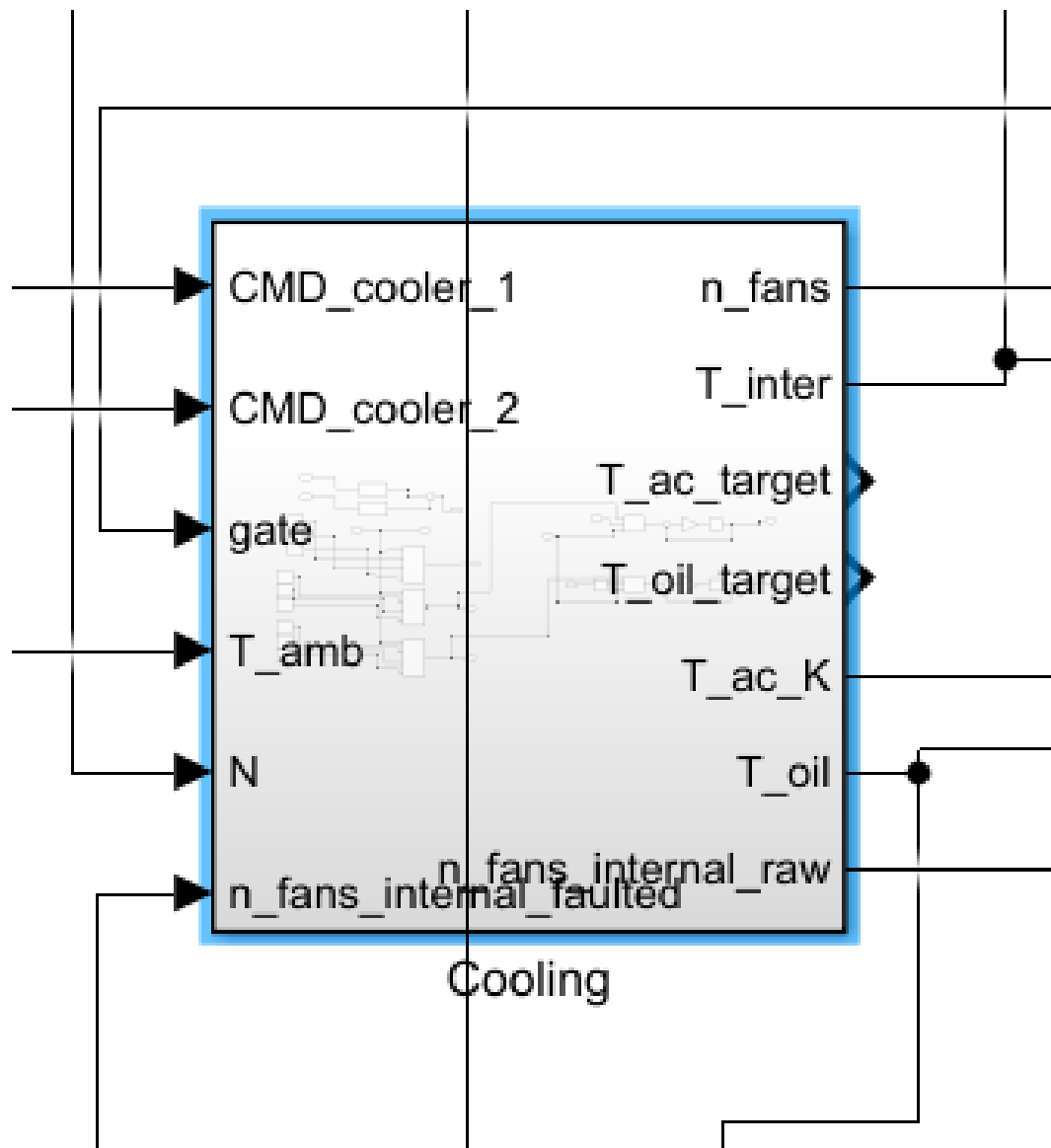


Figure 14: Cooling subsystem interface: cooler command inputs, fan-count output, and the resulting temperature outputs.

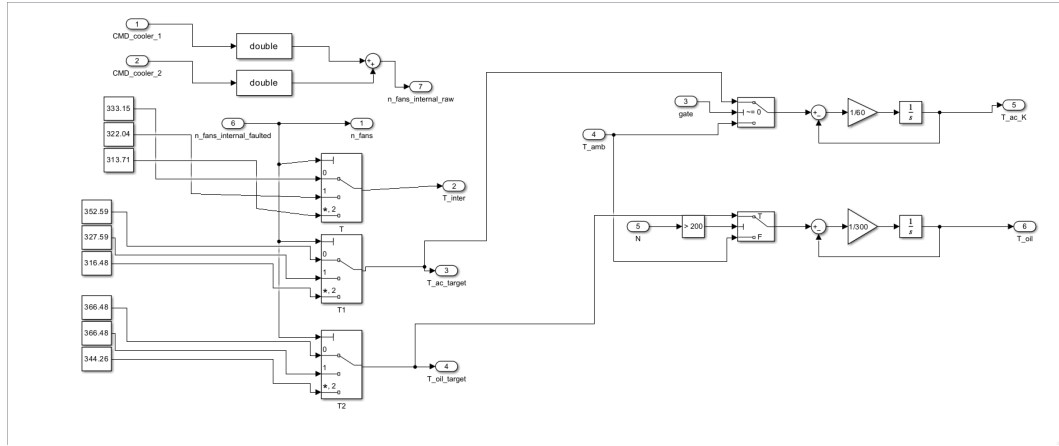


Figure 15: Cooling subsystem internals: fan-count derivation, lookup tables, and first-order lag states.

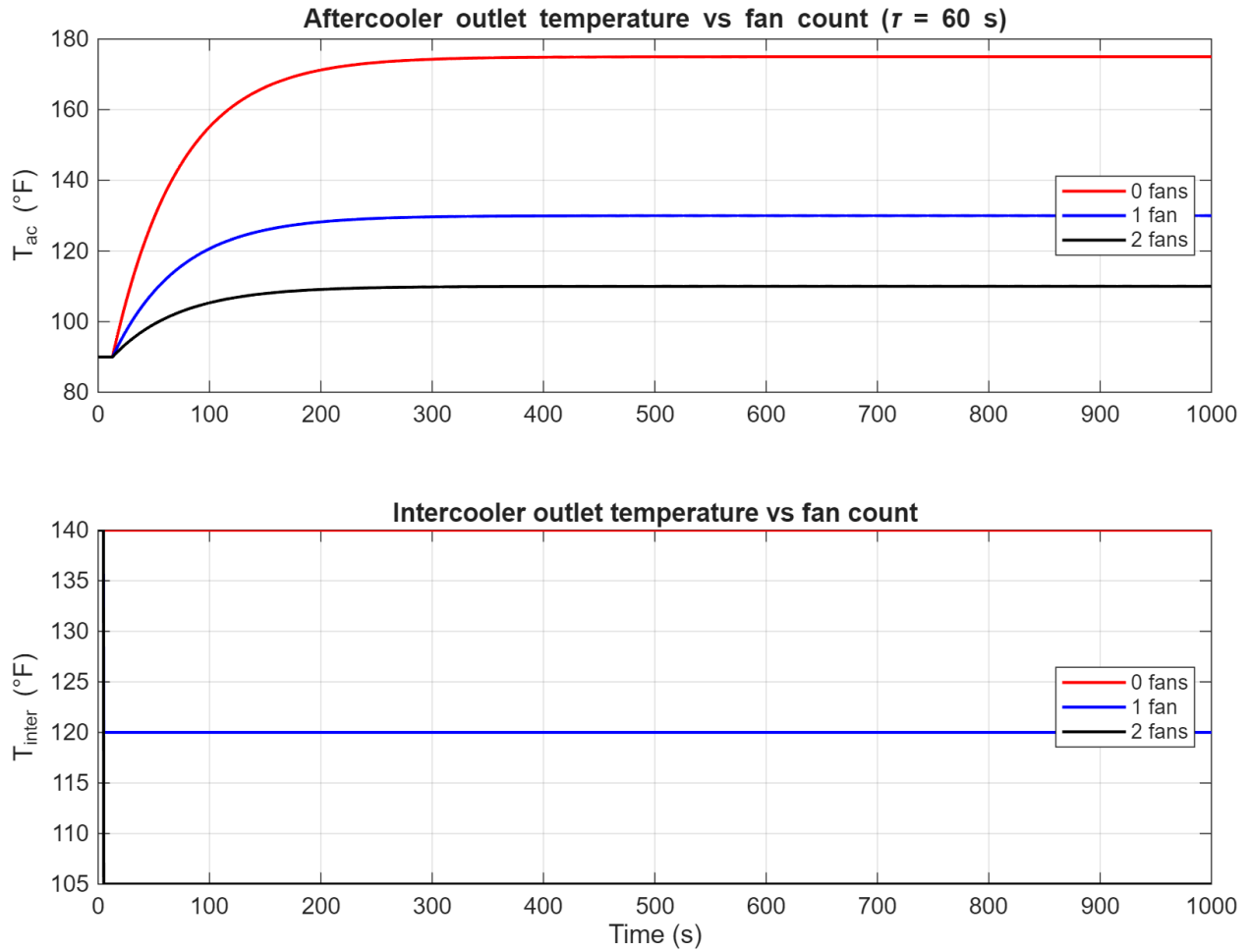


Figure 16: Aftercooler and intercooler outlet temperature response for 0/1/2 fans running (eq. (16)). Note the intercooler trace is a direct lookup with no first-order lag applied (unlike the aftercooler), so it steps rather than curves — this is correct model behavior, not a missing dynamic.

## 0.5 Lubrication

Lubrication is implemented as two separate subsystems, `Lube_Oil_Pressure` and `Engine_Oil_Temp`, sharing a single `CMD_aux_lube` OPC UA override mux at their common entry point.

- **Lube\_Oil\_Pressure** selects between a speed-derived running target and a fixed prelube target, with the prelube target gated by `CMD_aux_lube`. From `params.m`: running target  $P_{oil,run} = 8.27 \times 10^5$  Pa (120 psig), prelube target  $P_{oil,prelube} = 3.79 \times 10^5$  Pa (55 psig), lag time constant  $\tau_{oil,p} = 3$  s.
- **Engine\_Oil\_Temp** selects between an operating temperature and ambient temperature based on engine state. From `params.m`: running target  $T_{oil,run} = 190^\circ\text{F}$ , lag time constant  $\tau_{oil} = 400$  s.

Both subsystems terminate in a first-order lag to their respective outputs.

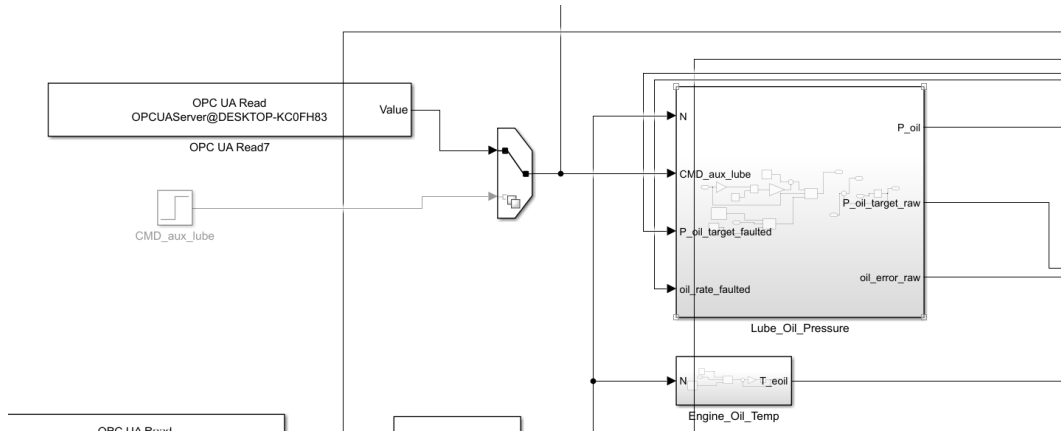


Figure 17: Lubrication interface context: the shared `CMD_aux_lube` OPC UA override mux feeding both lubrication subsystems.

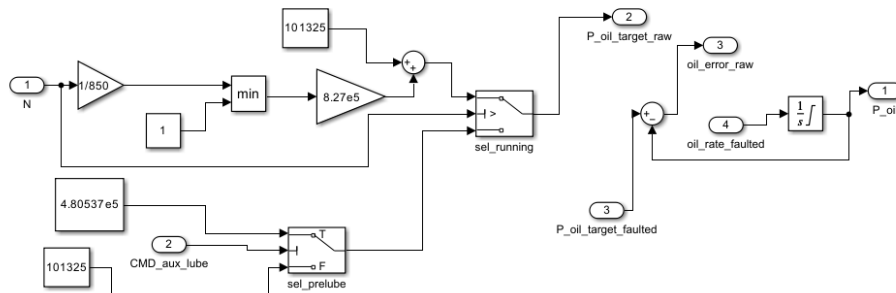


Figure 18: `Lube_Oil_Pressure` internals: running/prelube target selection into a first-order lag.

This dip is consistent with the acceptance criteria already checked by the model's verification suite (Section 0.10.5).

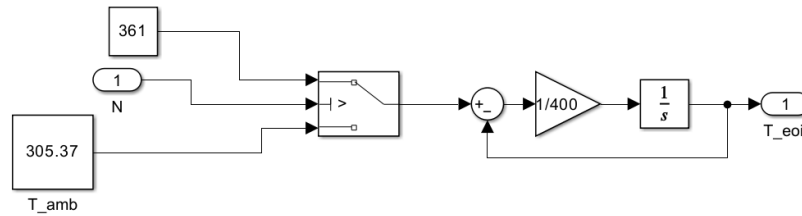


Figure 19: Engine\_Oil\_Temp internals: operating/ambient target selection into a first-order lag.

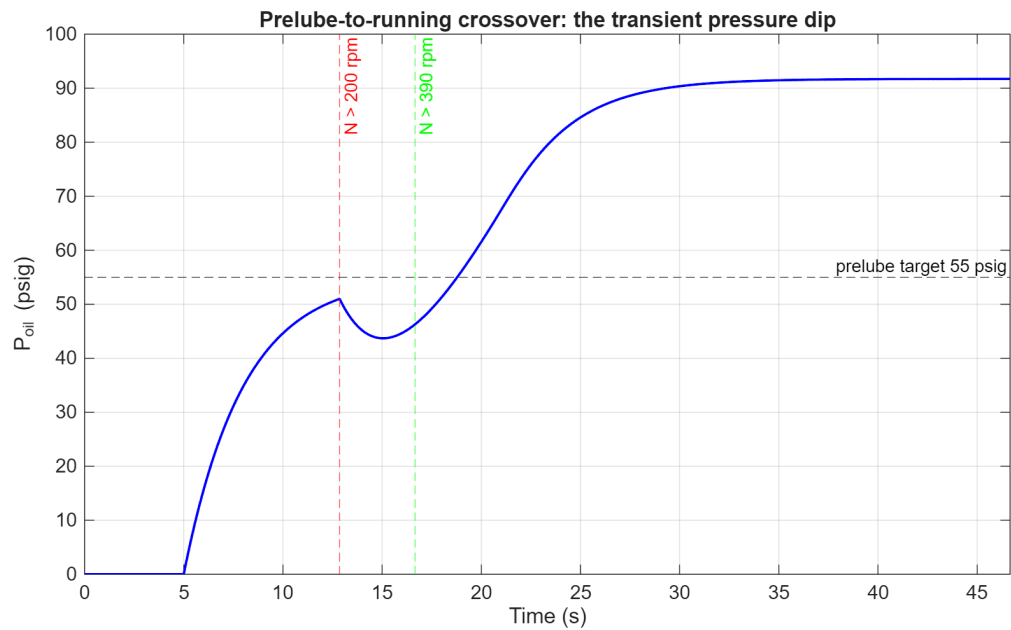


Figure 20: The transient lube oil pressure dip as the running-target formula briefly undercuts the prelube target during the crossover, per eq. (15).

## 0.6 Instrumentation

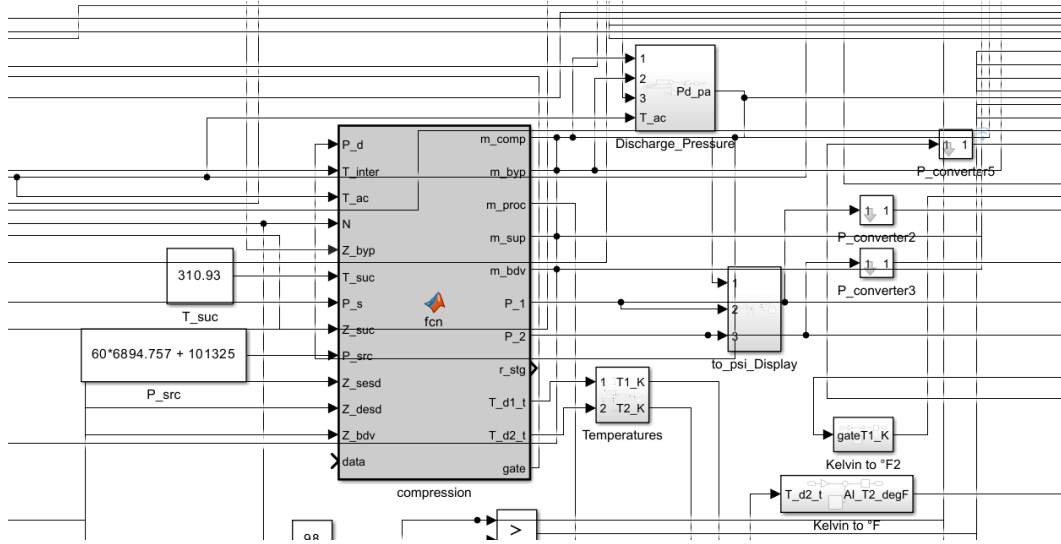


Figure 21: Instrumentation subsystem context: lead figure showing all instrument sub-blocks and their signal sources.

### Discharge Pressure

**Discharge\_Pressure** is a first-principles mass-balance pressure buildup model, not merely a unit converter. Three flow terms (compressor discharge flow, bypass return flow, process delivery flow) sum into a net mass-flow balance, which is scaled by the aftercooler temperature  $T_{ac}$  and integrated to produce the discharge pressure state. This distinction is explicit: unlike the converter blocks described below, **Discharge\_Pressure** carries real system dynamics, not a static transformation.

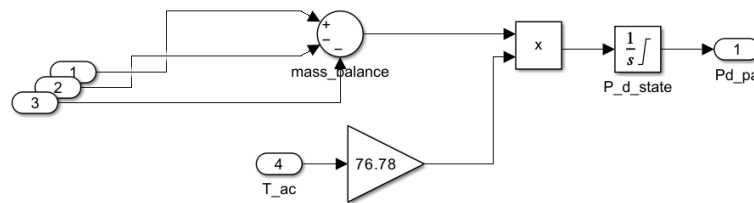


Figure 22: Discharge\_Pressure internals: three flow terms summed, scaled by  $T_{ac}$ , and integrated.

Governing flow coefficients from `params.m`:  $K_{suc} = 2.10 \times 10^{-3}$  (source to suction),  $K_{byp} = 1.50 \times 10^{-3}$  (discharge to suction via bypass),  $K_{proc} = 1.335 \times 10^{-4}$  (discharge to process via discharge ESD),  $K_{bdv} = 4.00 \times 10^{-3}$  (suction to atmosphere via blowdown). Converged acceptance values at the

design point (per the model’s verification suite): suction pressure  $\approx 29.8$  psig, first-stage discharge  $\approx 117.3$  psig, second-stage discharge  $\approx 377.2$  psig, final discharge  $\approx 1149.0$  psig, at a compressor mass flow of  $\approx 0.9448$  kg/s with the three stage ratios consistent at  $\approx 2.965$  each.

## Temperatures

**Temperatures** runs two parallel first-order lag chains, one per compression stage, using a common cylinder-temperature lag constant  $\tau_{T_{cyl}} = 45$  s from `params.m`.

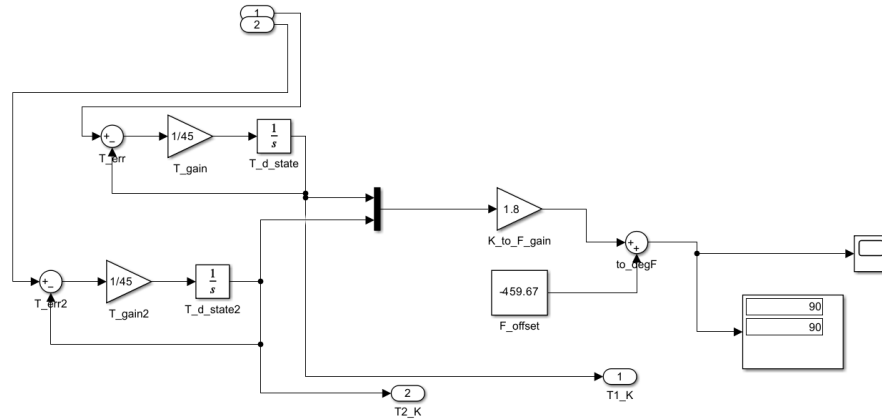


Figure 23: Temperatures internals: two parallel first-order lag chains.

These traces match the acceptance criteria already checked by the model’s verification suite (Section 0.10.5).

## Reusable Converter Blocks

**Kelvin to F** and **P\_converter** ( $\text{Pa} \rightarrow \text{psig}$ ) are reusable masked atomic converter blocks, each instantiated multiple times throughout the model. Each instance carries independently configurable saturation range limits set through block mask parameters — the same reuse pattern used by the valve family in Section 0.2, applied one abstraction layer up (a converter block rather than a full actuator).

Transmitter saturation ranges from `params.m`: suction 0–60 psig, stage 1 0–300 psig, stage 2 0–1000 psig, stage 3/final 0–2000 psig, oil 0–200 psig, speed 0–2000 rpm, temperature 0–500°F.



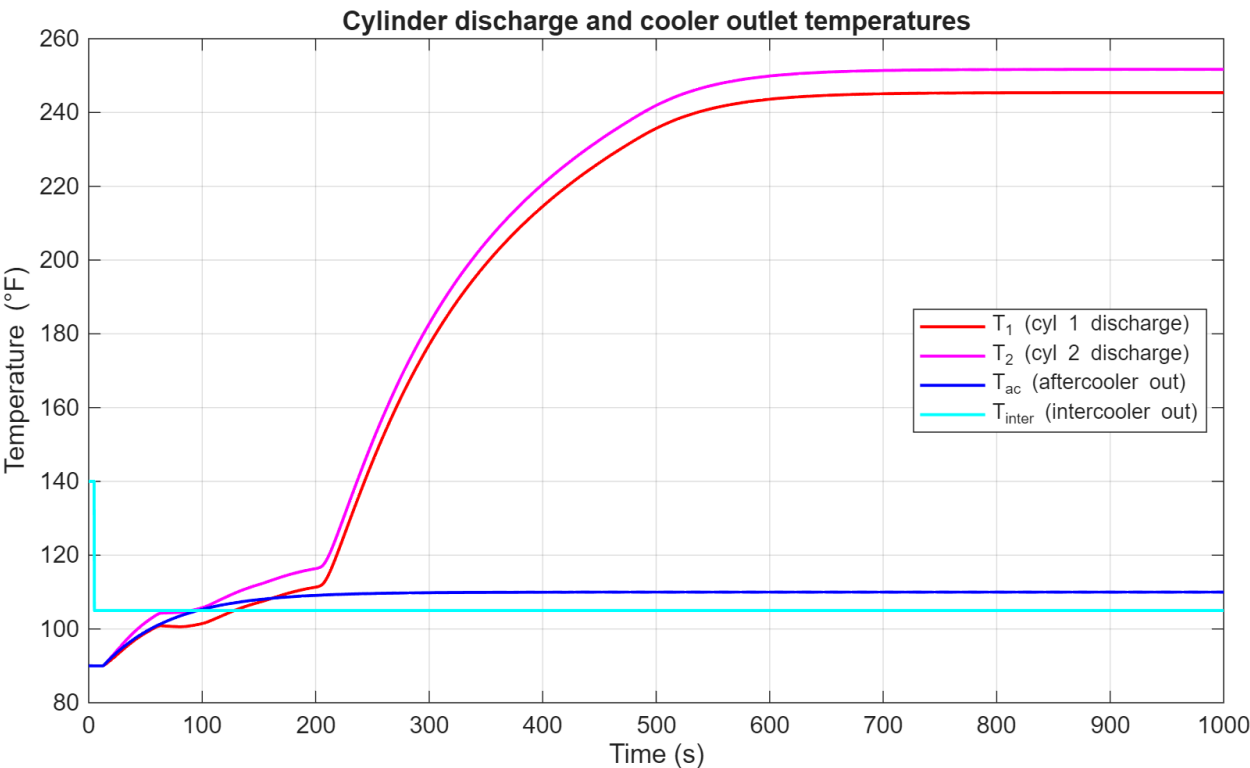


Figure 24: Cylinder discharge and cooler outlet temperatures over a full startup, showing why cylinder 2 (fed by the intercooler) is more sensitive to fan loss than cylinder 1 (eq. (11)).

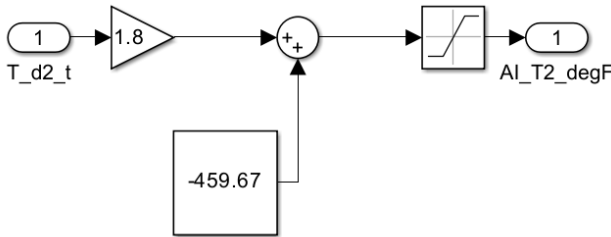


Figure 25: Kelvin to F reusable masked converter block internals.

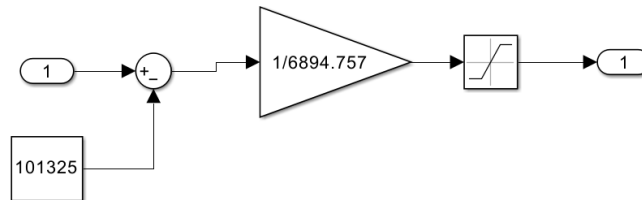


Figure 26: P\_converter (Pa to psig) reusable masked converter block internals.

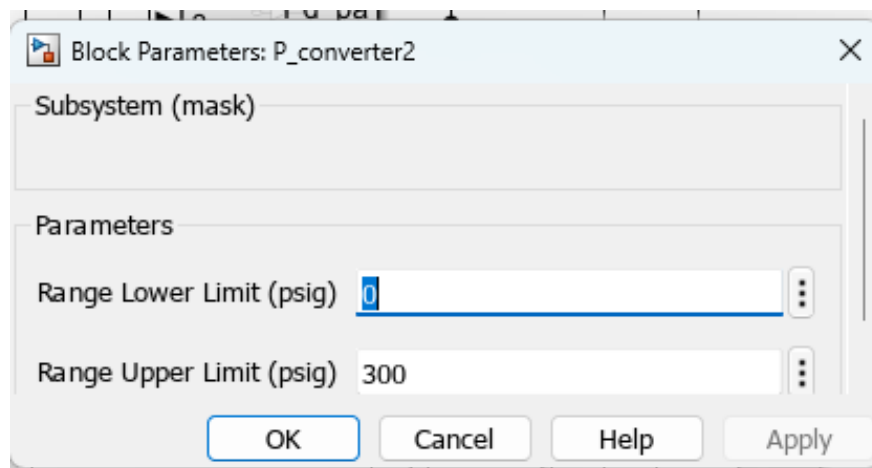


Figure 27: P\_converter mask parameters dialog: per-instance saturation range limits.

## 0.7 Fault Injection

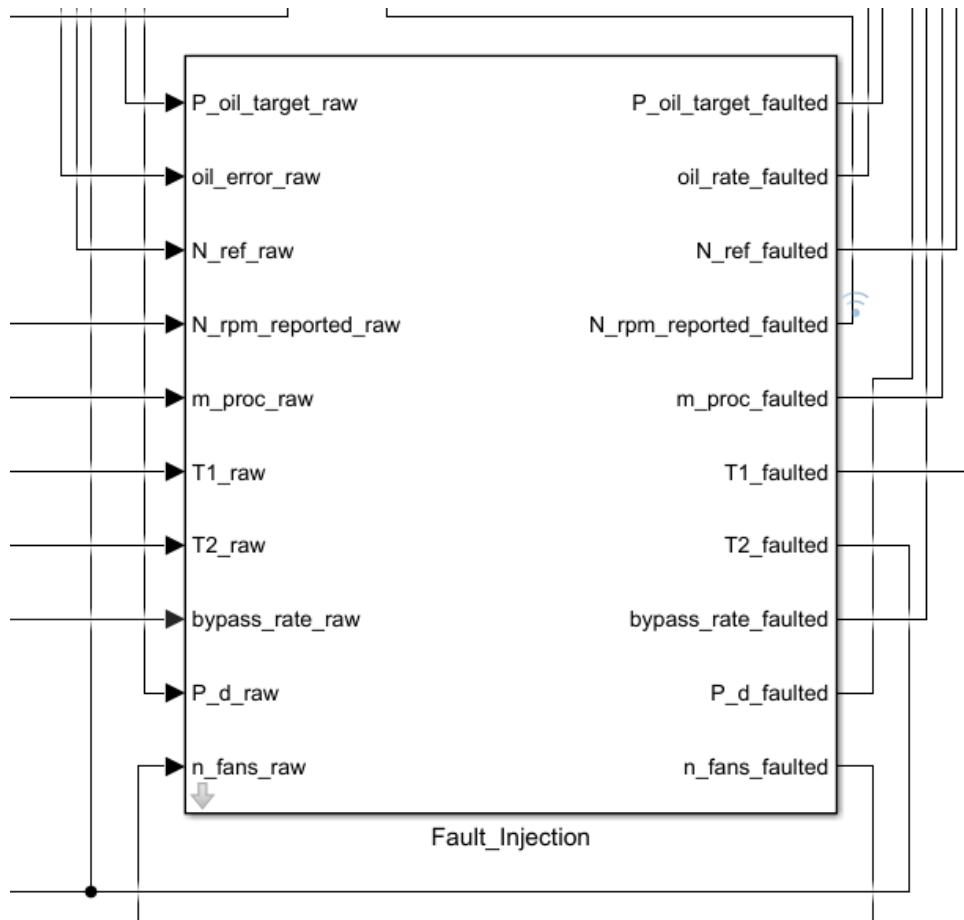


Figure 28: Fault Injection subsystem interface: the ten raw  $\rightarrow$  faulted signal pairs.

The subsystem implements ten raw  $\rightarrow$  faulted signal pairs, selectable through a checkbox mask menu that serves as the operator-facing fault selection UI (Figure 29). Most faults follow one idiom: a boolean flag gates a switch that selects between the raw signal and a substituted fault value or expression.

Internally, the faulted signal groups are organized as shown in Figures 30–33: oil pressure, oil rate, and speed-reference faults; RPM/overspeed and blocked-discharge faults; cylinder-temperature-bias and stuck-bypass faults; and signal-freeze and cooler-trip faults. The faulted engine speed signal (`N_rpm_reported_faulted`) is the value actually presented to the PLC across the OPC UA boundary (Section 0.8), consistent with the fault-injection design intent of testing sequencer response to corrupted feedback.

Two faults deviate from the value-substitution idiom and are called out explicitly:

- **P\_d\_faulted (signal freeze).** This fault changes the signal's *dynamics*, not its value: a sample-and-hold mechanism captures and holds the discharge pressure signal at whatever value it had when the fault was triggered, rather than substituting a fixed or computed replacement value. This is the only fault in the subsystem that alters signal dynamics rather than signal

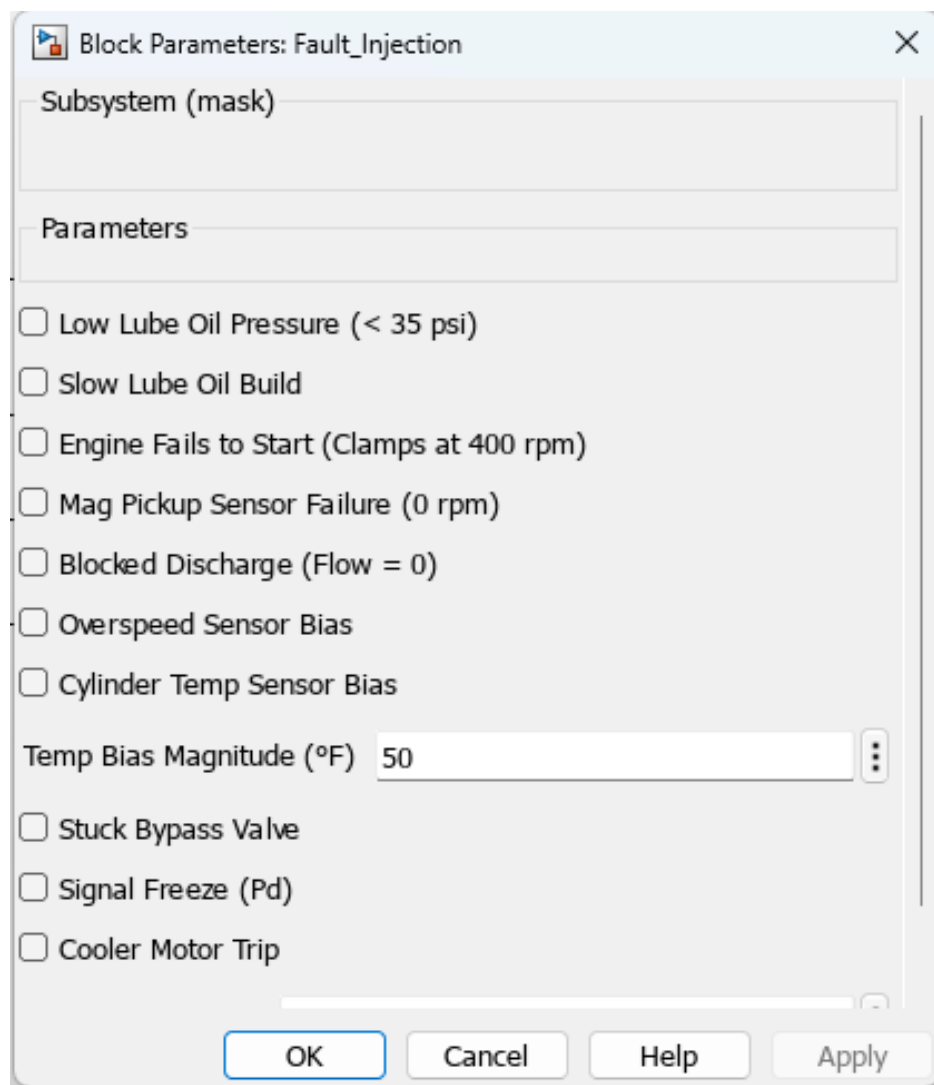


Figure 29: Fault Injection mask parameters: the operator-facing checkbox fault-selection menu.

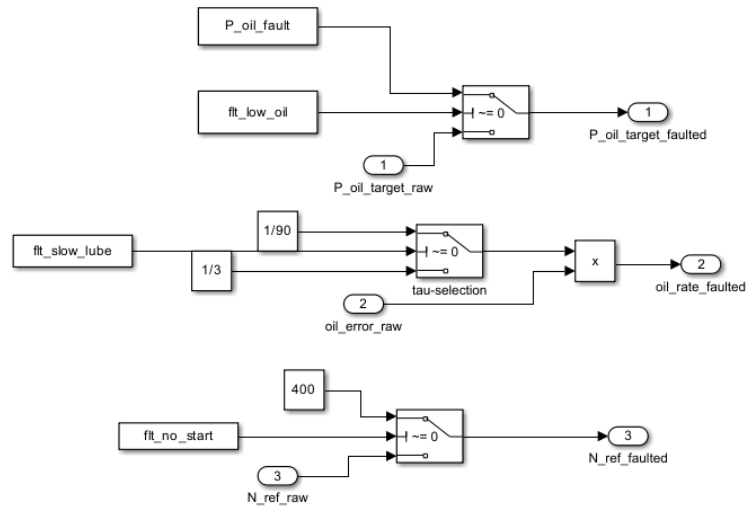


Figure 30: Fault Injection internals: oil pressure, oil rate, and speed-reference (N\_ref) faults.

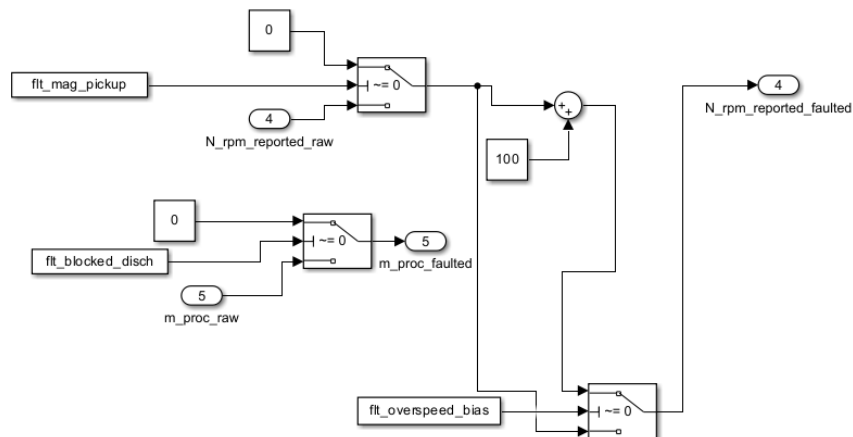


Figure 31: Fault Injection internals: RPM/overspeed and blocked-discharge faults.

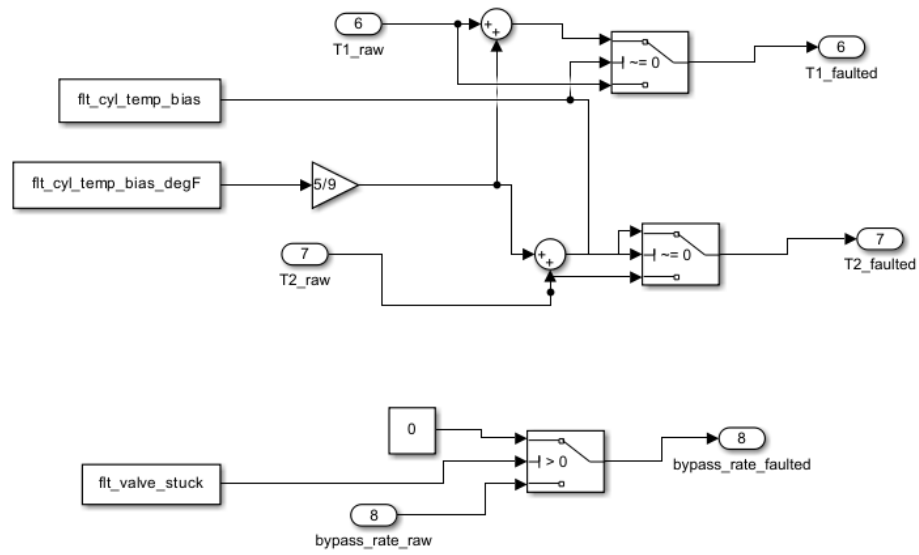


Figure 32: Fault Injection internals: cylinder temperature bias and stuck-bypass faults.

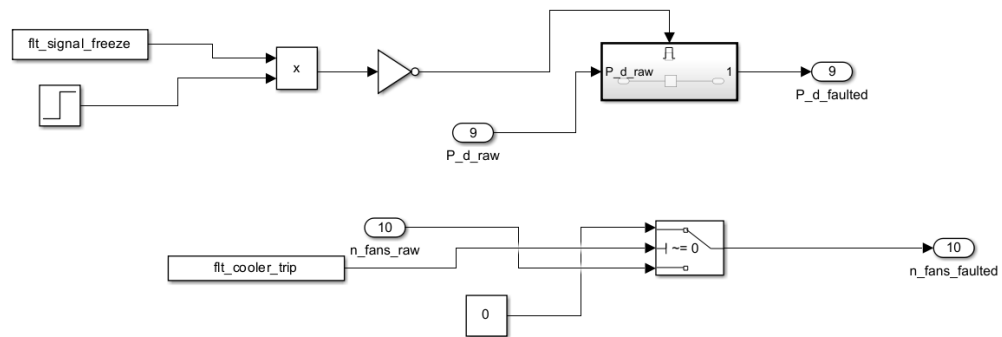


Figure 33: Fault Injection internals: signal-freeze and cooler-trip faults.

value.

- **oil\_rate\_faulted (tau-selection)**. This fault implements a time-constant selection sub-pattern: it switches the effective lag time constant applied to the oil rate signal between 1/90 and 1/3, rather than substituting a value or expression directly.

## 0.8 Logging vs. the True OPC UA Boundary

Logging and the OPC UA boundary are distinct concepts and must not be conflated. The Logging subsystem taps approximately 33 internal signals throughout the model; every tapped signal feeds a **To Workspace** block that supplies the MATLAB verification suite. Only a subset of those 33 logged signals also feeds an **OPC UA Write** block to CODESYS.

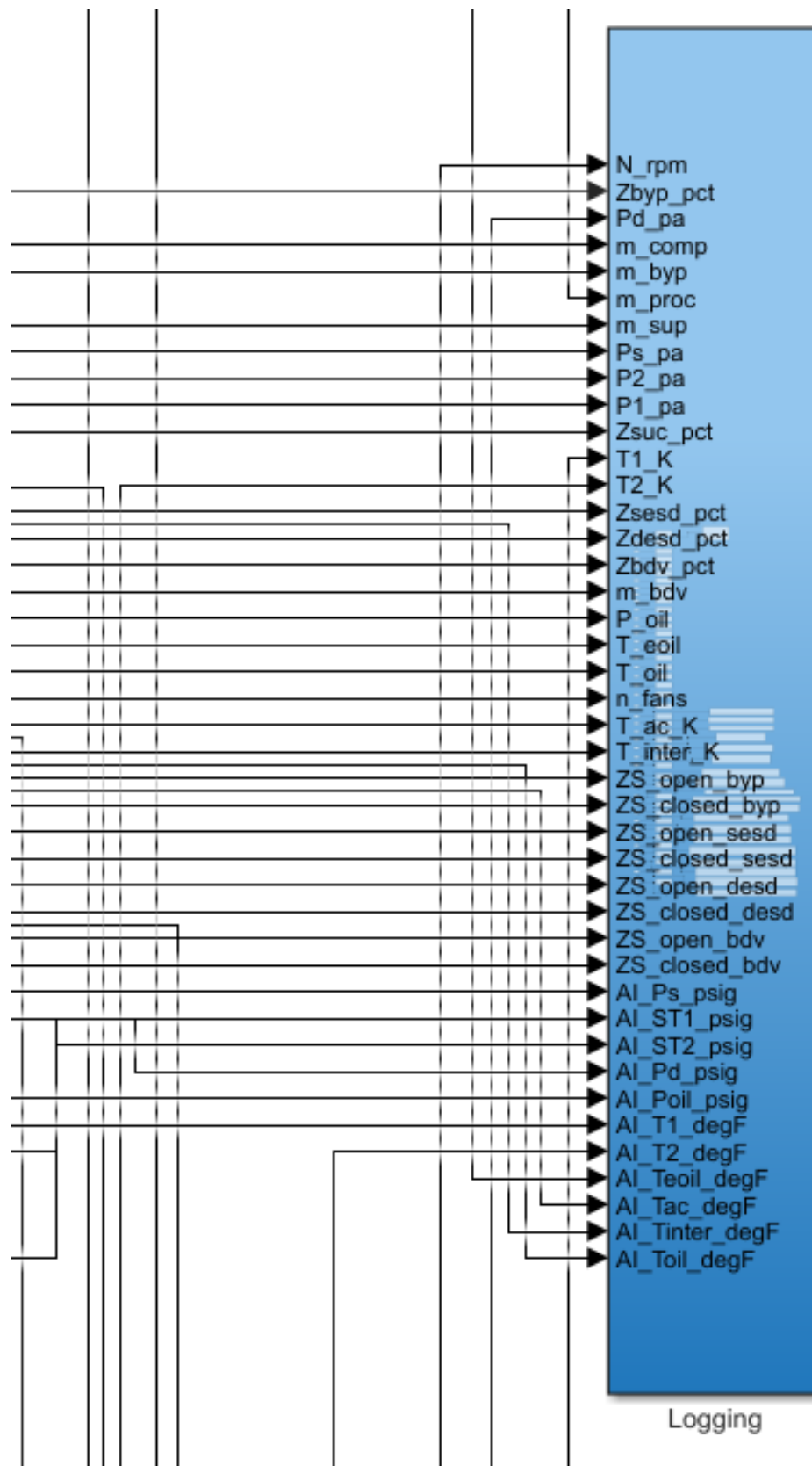


Figure 34: Logging subsystem interface: the full internal telemetry set tapped for verification.



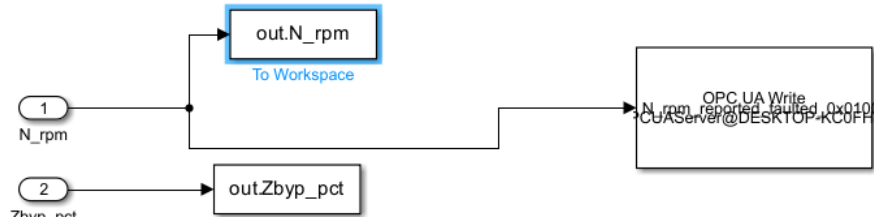


Figure 35: Logging dual-tap template: one internal signal feeding both a **To Workspace** block and, for the subset that crosses the OPC UA boundary, an **OPC UA Write** block. This pattern repeats once per tapped signal.

The true PLC-facing signal boundary is defined exclusively by **GVL OPCUA**: 13 commands PLC → model, 20 outputs model → PLC. Logging is the internal superset used for verification; **GVL OPCUA** is the authoritative external interface. The full listing is reproduced below verbatim as Listing 1.

**Listing 1.** **GVL OPCUA** — the authoritative PLC/model signal boundary, reproduced verbatim.

```

VAR_GLOBAL
// ---- PLC -> model commands (13) ----
CMD_cat_start      : BOOL;
CMD_driven_ready   : BOOL;
CMD_idle_rated     : BOOL;
CMD_cat_esd        : BOOL;
AO_speed           : REAL;
CMD_byp            : REAL;
CMD_suc            : REAL;
CMD_sesd           : BOOL;
CMD_desd           : BOOL;
CMD_bdv            : BOOL;
CMD_aux_lube       : BOOL;
CMD_cooler_1       : BOOL;
CMD_cooler_2       : BOOL;

// ---- model -> PLC outputs (20) ----
AI_Ps_psig         : REAL;
AI_ST1_psig        : REAL;
AI_ST2_psig        : REAL;
AI_Pd_psig         : REAL;
AI_Poil_psig       : REAL;
AI_T1_degF         : REAL;
AI_T2_degF         : REAL;
AI_Teoil_degF      : REAL;
AI_Tac_degF        : REAL;
AI_Tinter_degF     : REAL;
AI_Toil_degF       : REAL;
N_rpm_reported_faulted : REAL;

ZS_open_byp        : BOOL;
ZS_closed_byp      : BOOL;
ZS_open_sesd       : BOOL;
ZS_closed_sesd     : BOOL;
ZS_open_desd       : BOOL;
ZS_closed_desd     : BOOL;

```

```

ZS_open_bdv      : BOOL;
ZS_closed_bdv    : BOOL;

n_fans_out       : INT;
END_VAR

```

## 0.9 OPC UA Interface Mechanics

Each of the 33 signals declared in `GVL OPCUA` (13 inputs, 20 outputs) is served by its own individual OPC UA block instance in Simulink: one OPC UA Read block per input signal, one OPC UA Write block per output signal. This is 33 individual block instances total — the interface is not implemented as a single batched bus read or write. All OPC UA Read and Write blocks execute at a fixed 0.1 s sample time.

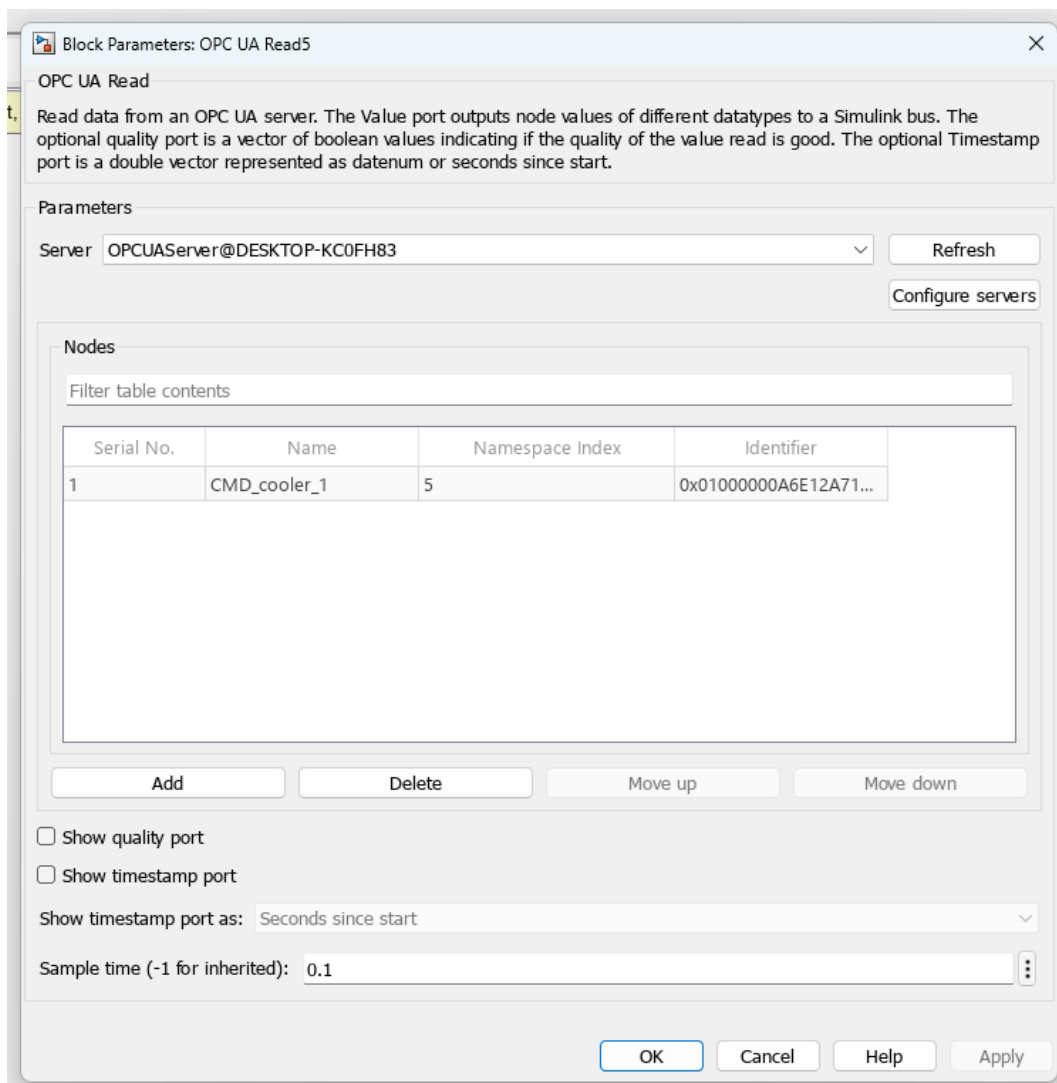


Figure 36: OPC UA Read block parameters: one node per block instance.

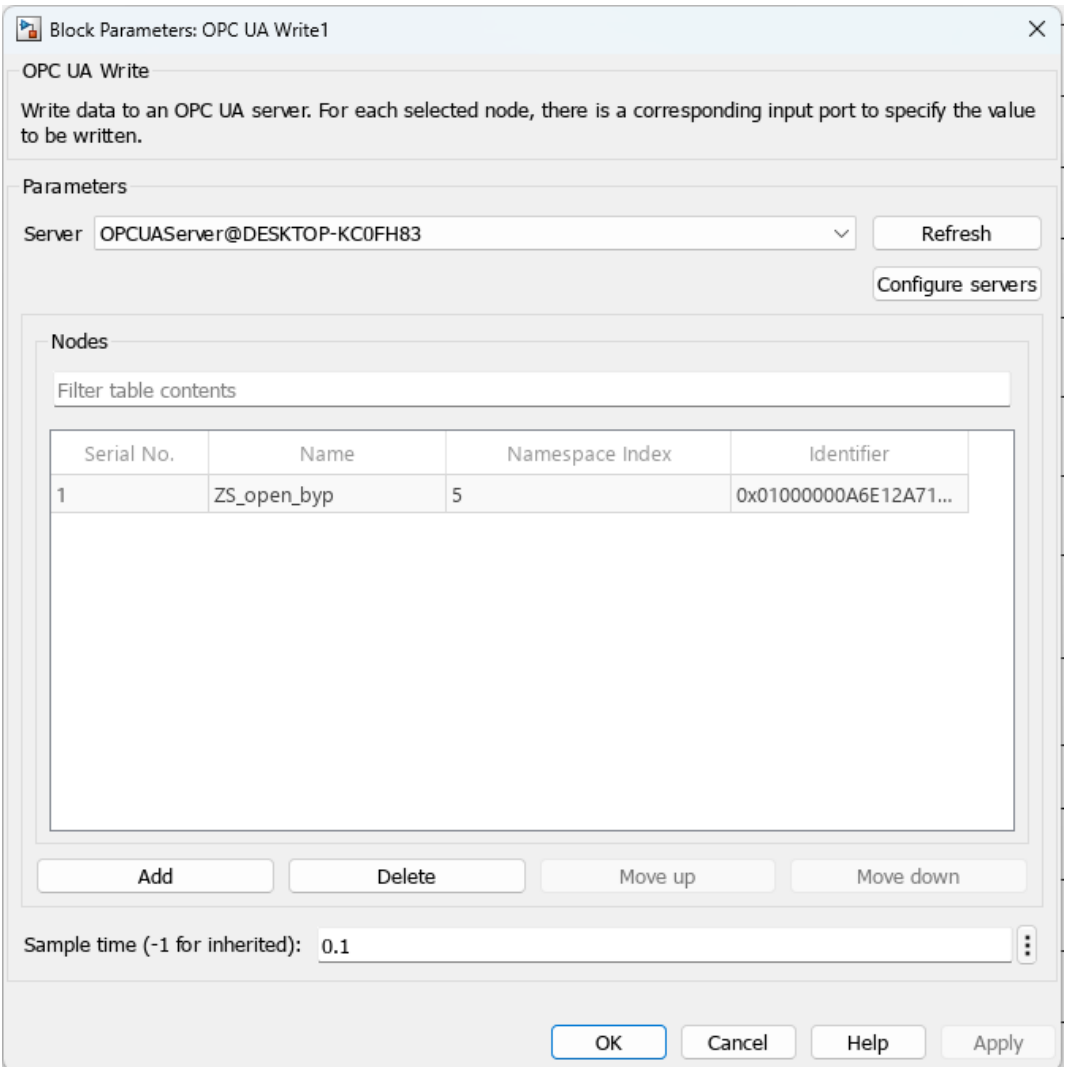


Figure 37: OPC UA Write block parameters: one node per block instance.

### 0.9.1 Variant Source Rationale

An earlier implementation selected between the live OPC UA input and a local test input using ordinary Switch blocks. This caused the OPC UA Read block to execute on every simulation step regardless of the `USE_OPCUA` flag state, because a Switch block evaluates both of its input branches unconditionally — it only discards one of them after the fact. With the OPC UA Read block executing every step whether or not its output was actually used, simulation performance degraded significantly.

The fix replaces the Switch block with a **Variant Source** block, configured with **Variant activation time** = **update diagram** and port conditions `USE_OPCUA == 1` / `USE_OPCUA == 0` (Figure 38). A Variant Source resolves its active branch at diagram-update time and compiles the unused branch out of the compiled model entirely, rather than evaluating both branches every step. This is a deliberate design decision made for a stated performance reason, not a stylistic preference: with the Switch-block implementation, the OPC UA Read executed unconditionally; with the Variant Source, the OPC UA Read branch is entirely absent from the compiled model whenever `USE_OPCUA == 0`.

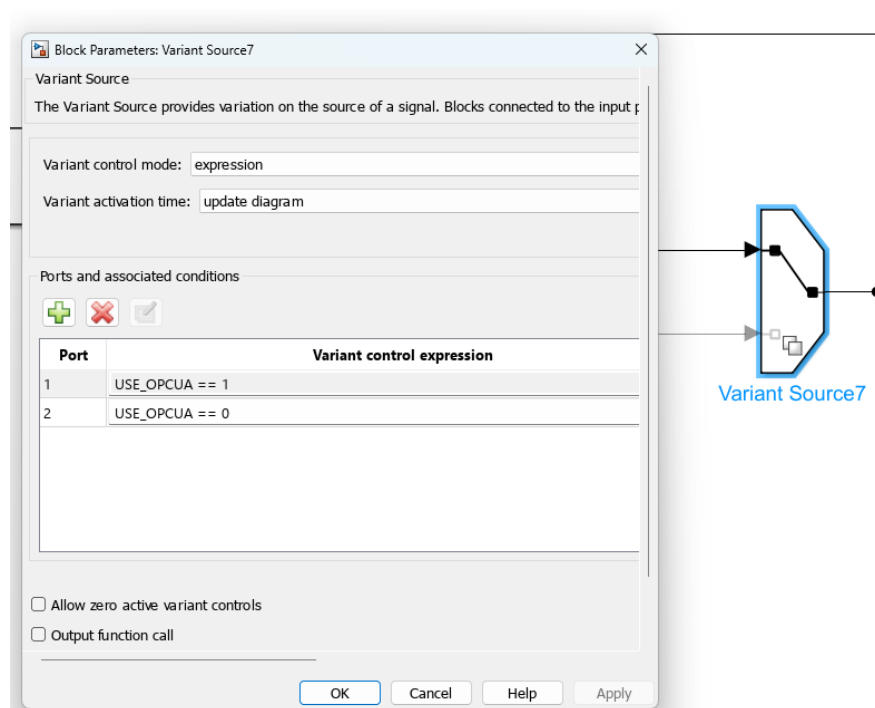


Figure 38: Variant Source block parameters: activation time = update diagram, port conditions on `USE_OPCUA`.

## 0.10 PLC Sequencer

The CODESYS sequencer (`PLC_PRG`) is a single scan-cycle **CASE** state machine driven by four operator commands (`HMI_start`, `HMI_stop`, `HMI_usd`, `HMI_reset`) forced from the CODESYS Watch window, plus process feedback read back from the model over `GVL_OPCUA`.

### 0.10.1 State Diagram

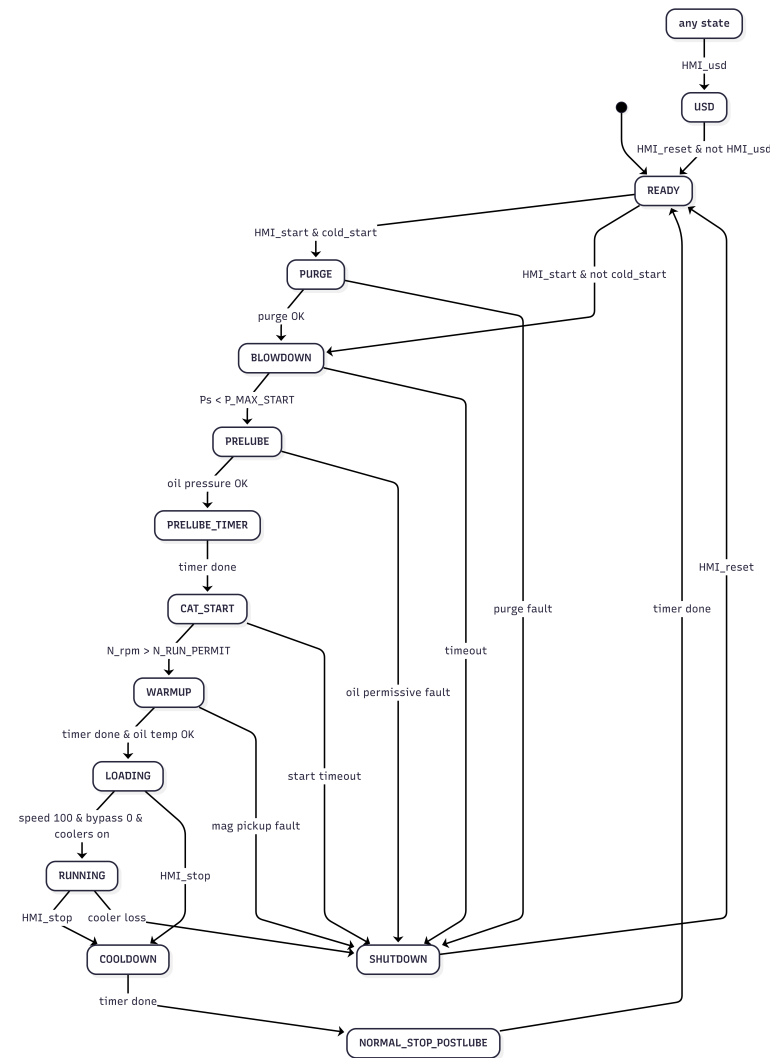


Figure 39: PLC sequencer state diagram. The “any state” node is a diagram-notation convenience representing the USD-override behavior (Unconditional Shutdown can preempt from any running step, per the unconditional priority check executed at the top of every scan cycle) — it is *not* an actual PLC step value.

At the top of every scan, before the **CASE** statement executes, the sequencer checks **HMI\_usd** unconditionally and forces **step := 200** if it is set and the machine is not already in step 200. This preemption is independent of whatever step the machine is currently executing, which is what the “any state” node in Figure 39 represents.

### 0.10.2 Step Table

| Step | Name                  | Description / Exit Condition(s)                                                                                                                                                                                                                                                                                                                                                                          |
|------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0    | READY                 | At-rest state. All commands de-energized except blowdown ( <b>CMD_bdv</b> = <b>TRUE</b> ); ramp outputs and fault message reset. Exit: on <b>HMI_start</b> , goes to step 10 if <b>cold_start</b> is set, otherwise to step 20.                                                                                                                                                                          |
| 10   | COLD START /<br>PURGE | Opens suction ESD, closes blowdown, drives suction target to the purge setpoint. After <b>T_BY_P_CLOSE</b> has elapsed, ramps bypass closed. Exit: on <b>T_PURGE</b> timeout, if suction pressure is below <b>P_PURGE_PERMIT</b> , faults to step 255 (“PURGE FAULT SHUTDOWN”); otherwise forces bypass to 100% and advances to step 20.                                                                 |
| 20   | BLOWDOWN              | Holds suction ESD open, closes blowdown, forces bypass and suction targets to zero. Exit: once suction pressure falls below <b>P_MAX_START</b> , opens the blowdown valve and advances to step 30; on <b>T_BLOWDOWN_MAX</b> timeout without reaching that pressure, faults to step 255 (“BLOWDOWN FAULT SHUTDOWN”).                                                                                      |
| 30   | PRELUBE               | Holds blowdown open, energizes the auxiliary lube pump. Exit: once oil pressure reaches <b>P_OIL_PERMIT</b> , advances to step 35; on <b>T_OIL_PERMIT</b> timeout, faults to step 255 (“COMPRESSOR PRELUBE PERMISSIVE FAULT”).                                                                                                                                                                           |
| 35   | PRELUBE<br>TIMER      | Holds auxiliary lube on, asserts the engine start command ( <b>CMD_cat_start</b> ). Exit: on <b>T_PRELUBE</b> timeout, asserts <b>CMD_driven_ready</b> and advances to step 40.                                                                                                                                                                                                                          |
| 40   | CAT START             | Asserts idle/rated command to the engine. Exit: once reported engine speed exceeds <b>N_RUN_PERMIT</b> , advances to step 50; on <b>T_START_MAX</b> timeout, faults to step 255 (“ENGINE FAILED TO START SHUTDOWN”).                                                                                                                                                                                     |
| 50   | WARM-UP               | Runs a fixed warm-up timer ( <b>T_B_TIMER</b> ). Exit: if reported engine speed drops to zero or below, faults immediately to step 255 (“MAG PICKUP FAULT SHUTDOWN”); otherwise once the timer expires and engine oil temperature reaches <b>T_OIL_LOAD</b> , advances to step 60.                                                                                                                       |
| 60   | LOADING               | Opens discharge ESD, ramps speed up to 100%, ramps bypass down to 0%, ramps suction target up to <b>SUC_RUN_SP</b> . Once engine speed reaches <b>N_COOLER</b> , energizes cooler 1 and, after <b>T_COOLER2</b> , cooler 2. Exit: once speed is at 100%, bypass is at 0%, and both coolers are confirmed on, advances to step 70 (RUNNING); at any point, <b>HMI_stop</b> diverts to step 80 (COOLDOWN). |

| Step | Name                      | Description / Exit Condition(s)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 70   | RUNNING                   | Holds speed at 100%, bypass at 0%, suction at <code>SUC_RUN_SP</code> . Continuously monitors both cooler statuses; loss of either faults immediately to step 255 (“LOSS OF COOLER MOTOR STATUS”). Exit: <code>HMI_stop</code> diverts to step 80.                                                                                                                                                                                                                                                                                      |
| 80   | COOLDOWN                  | Drops idle/rated command, ramps speed down to 0%, ramps bypass back up to 100% (reopening ahead of stop). Exit: on <code>T_COOLDOWN</code> timeout, drops the engine start command and advances to step 90.                                                                                                                                                                                                                                                                                                                             |
| 90   | NORMAL STOP<br>/ POSTLUBE | Closes suction ESD and discharge ESD, opens blowdown, de-energizes both coolers, energizes auxiliary lube (postlube). After <code>T_DER_DELAY</code> , drops <code>CMD_driven_ready</code> . Exit: on <code>T_POSTLUBE</code> timeout, de-energizes auxiliary lube, clears <code>cold_start</code> (next start is a warm start), and returns to step 0.                                                                                                                                                                                 |
| 200  | USD                       | Unconditionally reachable from any step via the priority check at the top of the scan (Section 0.10.1). De-energizes driven-ready, idle/rated, and engine start; asserts the engine ESD command; closes suction ESD, discharge ESD, and blowdown; forces bypass to 100%; de-energizes auxiliary lube and both coolers; zeroes speed and suction targets; sets <code>cold_start</code> so the next start is a cold start with a full purge. Exit: <code>HMI_reset</code> with <code>HMI_usd</code> no longer asserted returns to step 0. |
| 255  | PROCESS SHUT-<br>DOWN     | Fault-triggered shutdown state reached from steps 10, 20, 30, 40, 50, or 70. De-energizes driven-ready, idle/rated, and engine start; closes suction and discharge ESD; opens blowdown; de-energizes both coolers; energizes auxiliary lube; zeroes speed and suction targets; <code>cold_start</code> is cleared (not forced). Exit: <code>HMI_reset</code> de-energizes auxiliary lube and returns to step 0.                                                                                                                         |

The permissive-crossing times shown above are consistent with the acceptance criteria already checked by the model’s verification suite (Section 0.10.5).

### 0.10.3 Constants — Current Test Values

**These are CURRENT TEST VALUES.** They are short-duration timings chosen to prove the sequencer logic quickly during development, not final Operating Philosophy timings. The “Op Phil reference value” column has now been cross-checked against `PN17481 OpPhil#4.pdf` (Spartan Controls REM *Vue* 500S Operating Philosophy, Revision 4, January 2, 2008). Only constants with an explicit, unambiguously labeled corresponding value in that document are filled in, each with a footnote citing the source page/section; every other row is left blank because no matching numeric value could be found in the document — the majority of timer and setpoint constants are described only qualitatively there (e.g. “the *Cooldown Timer* is adjustable from the HMI”) with no default value stated.



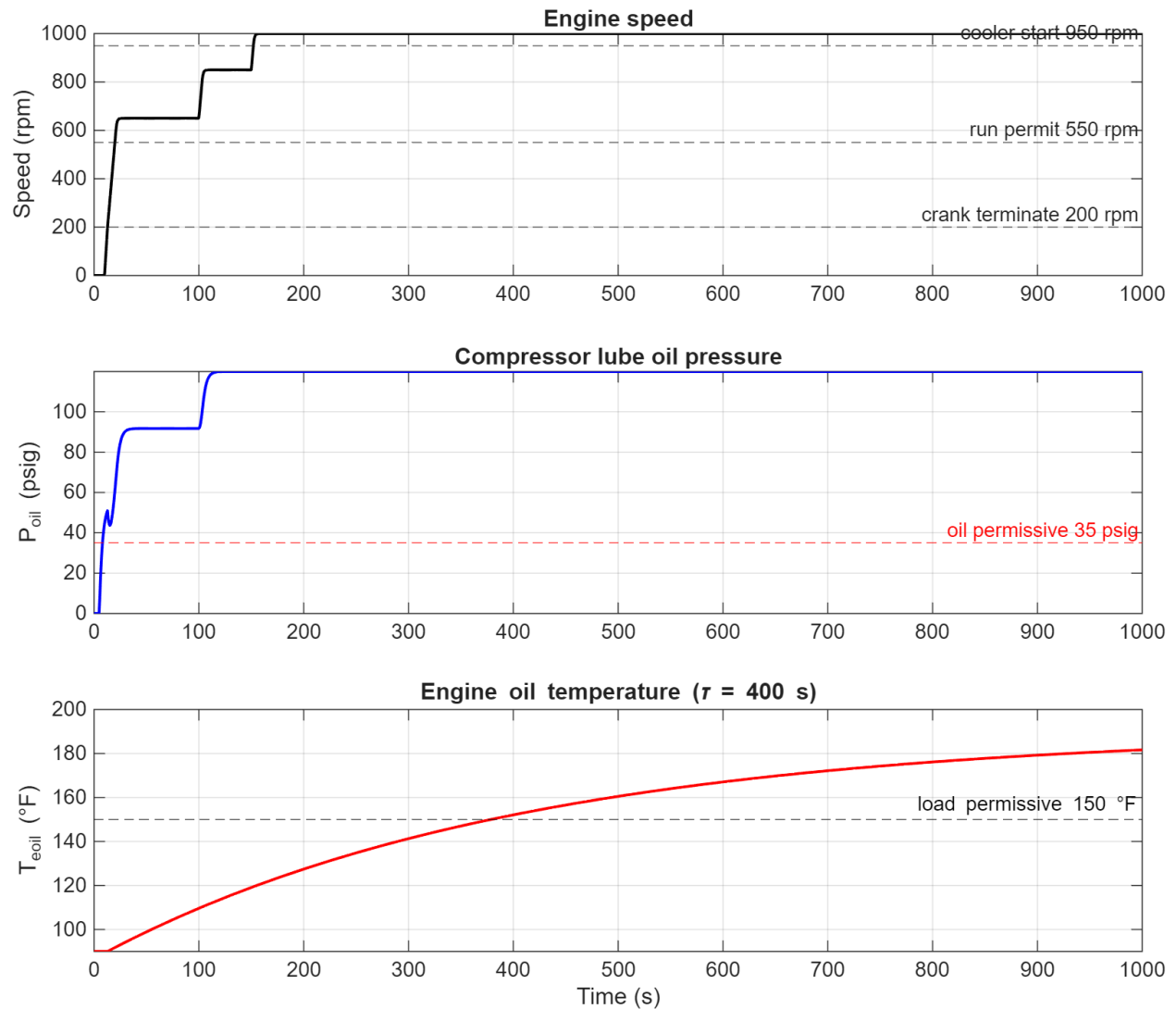


Figure 40: Engine speed, lube oil pressure, and engine oil temperature during startup, with the sequencer's permissive thresholds marked, showing why each permissive check must wait on a genuine physical transient. Compressor lube oil pressure settles at 119.9 psig, consistent with the model's acceptance criteria.

| Constant       | Test Value | Meaning                                             | Op Phil ref.<br>value |
|----------------|------------|-----------------------------------------------------|-----------------------|
| T_PURGE        | 50 s       | Cold-start purge timer                              |                       |
| T_BYP_CLOSE    | 5 s        | Delay before bypass begins closing during purge     | 30 s <sup>1</sup>     |
| T_BLOWDOWN_MAX | 45 s       | Blowdown step maximum time before fault             | 600 s <sup>2</sup>    |
| T_OIL_PERMIT   | 10 s       | Prelube oil-pressure permissive timeout             |                       |
| T_PRELUBE      | 5 s        | Prelube timer before engine start command           |                       |
| T_START_MAX    | 20 s       | Engine start maximum time before fault              |                       |
| T_B_TIMER      | 10 s       | Warm-up fixed timer                                 |                       |
| T_COOLER2      | 5 s        | Delay between cooler 1 and cooler 2 energization    | 20 s <sup>3</sup>     |
| T_COOLDOWN     | 15 s       | Cooldown step timer                                 |                       |
| T_POSTLUBE     | 5 s        | Postlube timer before return to READY               |                       |
| T_DER_DELAY    | 7 s        | Delay before dropping driven-ready in normal stop   | 7 s <sup>4</sup>      |
| P_MAX_START    | 10.0       | Suction pressure below which blow-down exits (psig) |                       |
| P_PURGE_PERMIT | 15.0       | Minimum suction pressure to pass purge (psig)       |                       |
| P_OIL_PERMIT   | 35.0       | Minimum oil pressure to exit prelube (psig)         |                       |
| T_OIL_LOAD     | 150.0      | Minimum engine oil temp to exit warm-up (°F)        |                       |
| N_RUN_PERMIT   | 550.0      | Minimum engine speed to exit CAT START (rpm)        |                       |
| N_MIN_LOAD     | 850.0      | Declared minimum-load speed constant (rpm)          | 850 rpm <sup>5</sup>  |
| N_COOLER       | 950.0      | Engine speed at which cooler 1 energizes (rpm)      | 950 rpm <sup>6</sup>  |
| SUC_PURGE_SP   | 90.0       | Suction valve purge target (%)                      |                       |

<sup>1</sup>OpPhil §2, Table 2-1, Item 2 (Cold Start / Purge), p. 6: “After 30 seconds the Bypass Valve will be ramped to the closed position.”

<sup>2</sup>OpPhil §2, Table 2-1, Item 2 (Warm Start / Blowdown), p. 6: “If after 10 minutes the suction pressure does not reach the Maximum Start Pressure setpoint a ‘BLOWDOWN FAULT SHUTDOWN’ will be generated.”

<sup>3</sup>OpPhil §2, Table 2-1, Item 4 (Engine Warm-up), p. 7: “20 seconds later the Cooler Motor #2 Run Command relay will energize.”

<sup>4</sup>OpPhil §2.1 (Compressor Cooldown and Stop), p. 8: “The CAT DRIVEN EQUIPMENT READY relay will de-energize seven seconds later.” Also stated identically in §2.3 (Engine / Compressor Shutdown), p. 9.

<sup>5</sup>OpPhil §4 (Automatic Loading Control), p. 13, Capacity Overview screen: Speed Control dial range labeled “850 – 1200 RPM,” with body text confirming “the engine speed setpoint changes from the Engine Minimum Load Speed to Engine Maximum Load Speed.”

<sup>6</sup>OpPhil §2, Table 2-1, Item 4 (Engine Warm-up), p. 7: “When the engine reaches the Cooler Motor Start Speed Setpoint (950 RPM), the Cooler Motor #1 Run Command relay will energize.”

| Constant   | Test Value | Meaning                          | Op Phil ref.<br>value |
|------------|------------|----------------------------------|-----------------------|
| SUC_RUN_SP | 45.0       | Suction valve running target (%) |                       |
| RATE_BYP   | 10.0       | Bypass ramp rate (%/s)           |                       |
| RATE_SPD   | 10.0       | Speed ramp rate (%/s)            |                       |
| RATE_SUC   | 20.0       | Suction valve ramp rate (%/s)    |                       |
| TS         | 0.02 s     | Task period                      |                       |

**Note on T\_B\_TIMER:** OpPhil §2, Table 2-1, Item 3 (CAT Start), p. 7 states “The 120 second B Timer starts” and “The 20 second b Timer starts,” referring to the Class B / Class b shutdown-bypass timers defined in Appendix B (p. 32), not to a warm-up fixed timer. Since this is not unambiguously the same constant as T\_B\_TIMER in this report’s model (which times the warm-up permissive, not a shutdown-class bypass window), it is left blank per the strict-match rule above rather than guessed.

#### 0.10.4 USD vs. Normal Stop

Both USD and normal stop originate from a running state, and both are nominally “stop” commands, but they drive the package to two different physical end states. USD (step 200) drives suction ESD closed and vents the package to near-zero discharge pressure, forcing a cold restart with a full purge cycle on the next start. Normal stop (steps 80 → 90) holds the bypass open through cooldown and settles the package near the design discharge pressure, allowing a warm restart without a purge cycle. The same nominal “stop” command therefore produces two different physical outcomes depending on which path — USD or normal stop — is taken to reach it.

The two settling values shown above match the acceptance criteria already checked by the model’s verification suite (Section 0.10.5).

#### 0.10.5 Verification

The model and sequencer are exercised by a MATLAB verification suite (`run_tests.m` plus a set of `verify_*.m` scripts covering valves, lubrication, cooling, instrumentation, fault injection, compression edge cases, parameter synchronization, acceptance criteria, and the USD/ normal-stop shutdown demonstration). At the time of this report, 34 self-tests across this suite pass.

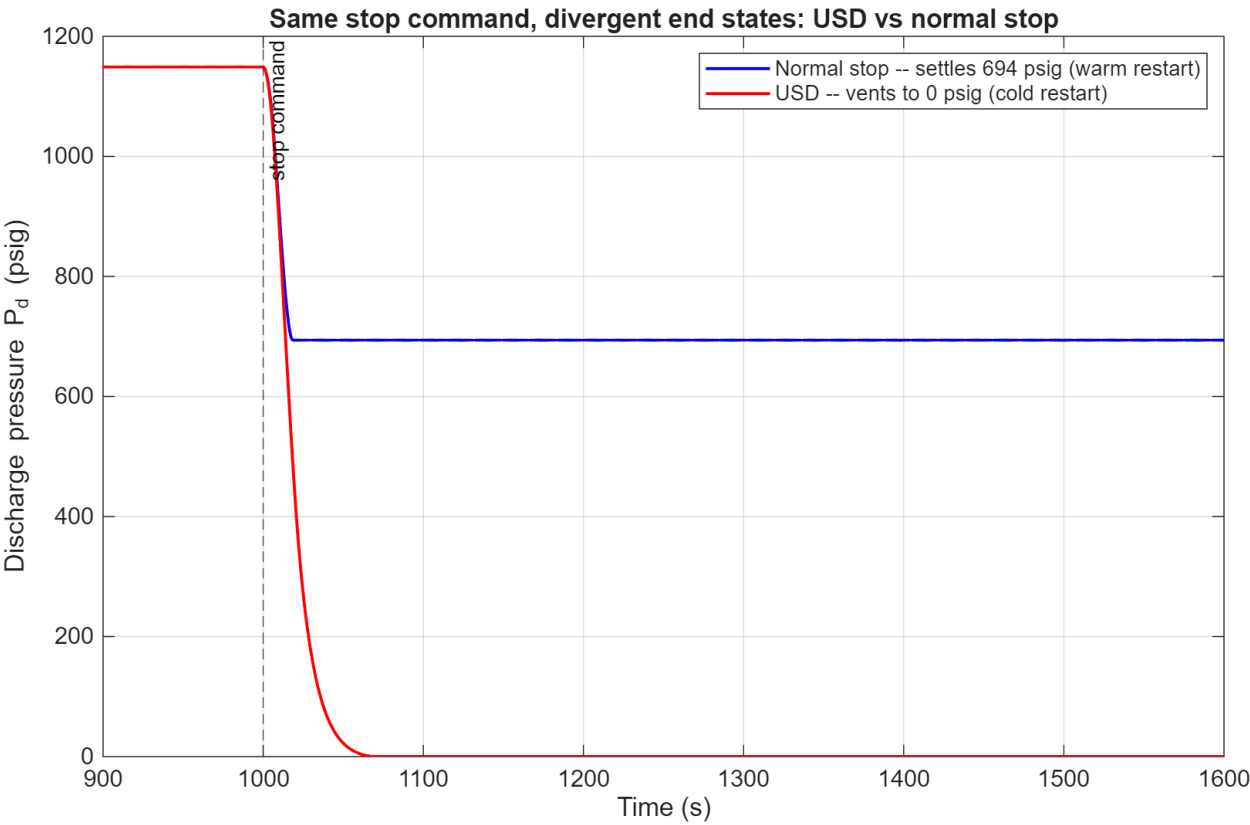


Figure 41: Discharge pressure following an identical stop command under normal stop and USD, demonstrating the divergent end states discussed above.

## Part III

# Environment Setup Guide

This part documents the procedure for reproducing the CODESYS/Simulink hardware-in-the-loop environment on a clean Windows machine. Steps are sequential and imperative; follow them in order on a first-time setup.

Simulink, with the Industrial Communication Toolbox (required for the OPC UA Read/Write blocks used to interface with the CODESYS soft-PLC), is assumed to already be installed via institutional/university MATLAB licensing. Its installation is not covered by this guide, since it is not a public free download like the two tools below.

## 0.1 CODESYS IDE and Control Win Runtime Installation

The CODESYS<sup>7</sup> IDE and the CODESYS Control Win runtime are separate installers. Installing the IDE alone does not provide a runtime to connect to.

1. Install the CODESYS Development System (the IDE) from the CODESYS Store/installer package.
2. Separately install **CODESYS Control Win**, the soft-PLC runtime that hosts the running program on the local machine. This is a distinct installable component, not a byproduct of the IDE installation.
3. Confirm the CODESYS Control Win service is running (system tray icon or Windows Services) before attempting to connect the IDE to a device.

## 0.2 Npcap Installation

1. Install Npcap<sup>8</sup> and, during setup, enable **WinPcap-compatible mode**.
2. Without WinPcap-compatible mode, the OPC UA server inside CODESYS registers internally but never opens a real TCP listener on the network interface — the server appears active in CODESYS but is unreachable from Simulink or any other OPC UA client on the network.

## 0.3 Project Creation

1. In the CODESYS IDE, create a new project using the **Standard Project** template.
2. Do not use the Empty Project template. Empty lacks Task Configuration and the Communication Manager, both of which are required later for cyclic task scheduling and the OPC UA server configuration.
3. Confirm the resulting device tree matches the standard layout (Device, PLC Logic, Task Configuration, etc.) before proceeding, as shown in Figure 42.

---

<sup>7</sup>Available from the CODESYS Store: <https://store.codesys.com> (international) or <https://us.store.codesys.com> (North America). One-time account registration is required. The installer bundles both the CODESYS Development System (IDE) and a demo version of the CODESYS Control Win SL SoftPLC runtime.

<sup>8</sup>Official installer, SDK, and source: <https://npcap.com/#download>. Free for use on up to 5 systems, including commercial use.

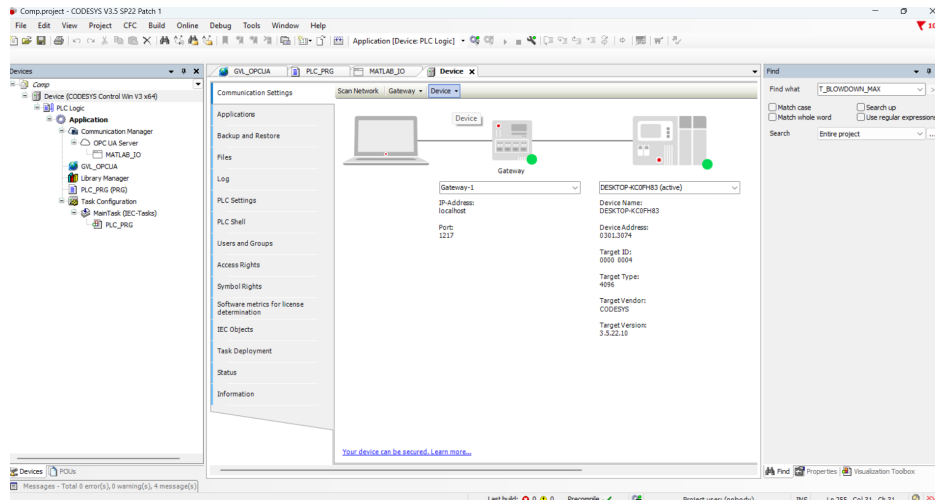


Figure 42: CODESYS Devices tree for a Standard Project, showing Task Configuration and Communication Manager present.

## 0.4 Symbol Publishing Configuration

1. Add exactly **one** Symbol Publishing object to the project, named **MATLAB\_IO**. This object controls which global variables are exposed as OPC UA nodes.
2. Every Standard Project is created with a default IEC Symbol Publishing object already present. This default object must be **removed, or emptied of all symbol selections** before adding **MATLAB\_IO**.
3. If both the default object and **MATLAB\_IO** publish overlapping symbols, the OPC UA address space build fails with duplicate-node errors. Only one Symbol Publishing object may be active with a non-empty symbol selection at a time.

## 0.5 Device Security Settings

1. Open the Device Security Settings for the CODESYS Control Win device (Figure 43).
2. Set **Communication Mode** = **ALL**. The default value restricts which communication channels are permitted and blocks external client connections, including the OPC UA client running on Simulink.
3. Set **Activation** = **ACTIVATED**. This also defaults to a state that blocks connections.
4. Both settings default to values that block connections. Both must be changed explicitly; the runtime will otherwise appear to start normally while refusing incoming connections.

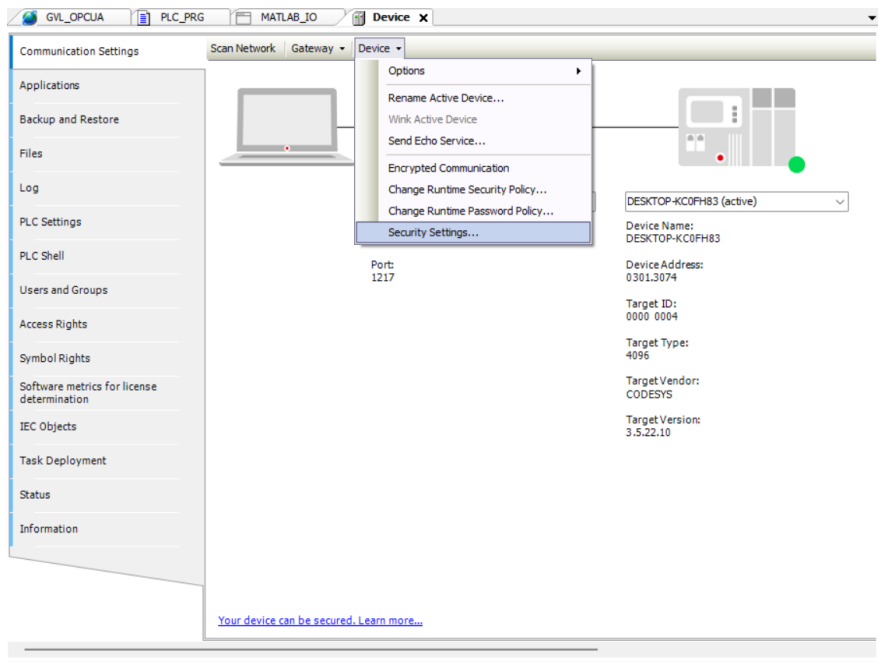


Figure 43: Menu location for Device Security Settings.

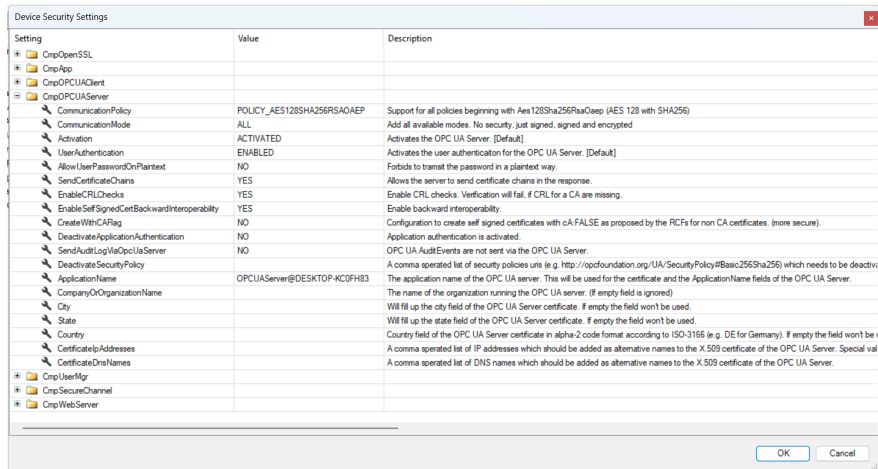


Figure 44: Device Security Settings dialog with the two required non-default values: Communication Mode = ALL, Activation = ACTIVATED.



## 0.6 Runtime Security Policy

1. Open the Runtime Security Policy configuration (Figure 45).
2. Set encryption to **Optional**.
3. Set **Code Signing** = **All**.
4. Set user management to **Optional**.
5. Enable **Allow anonymous login**.

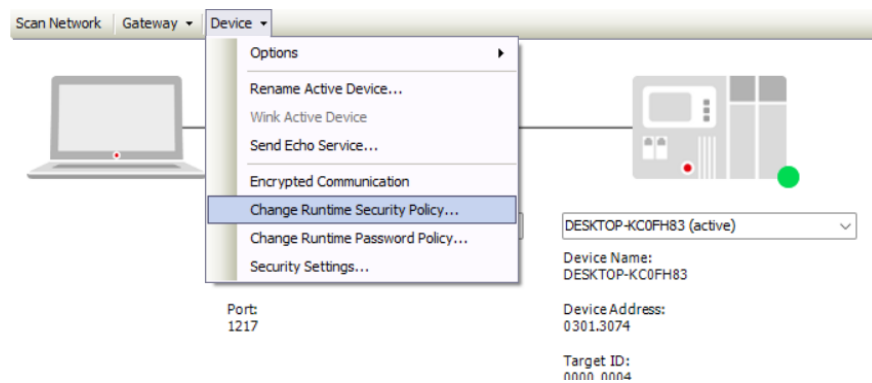


Figure 45: Menu location for the Runtime Security Policy.

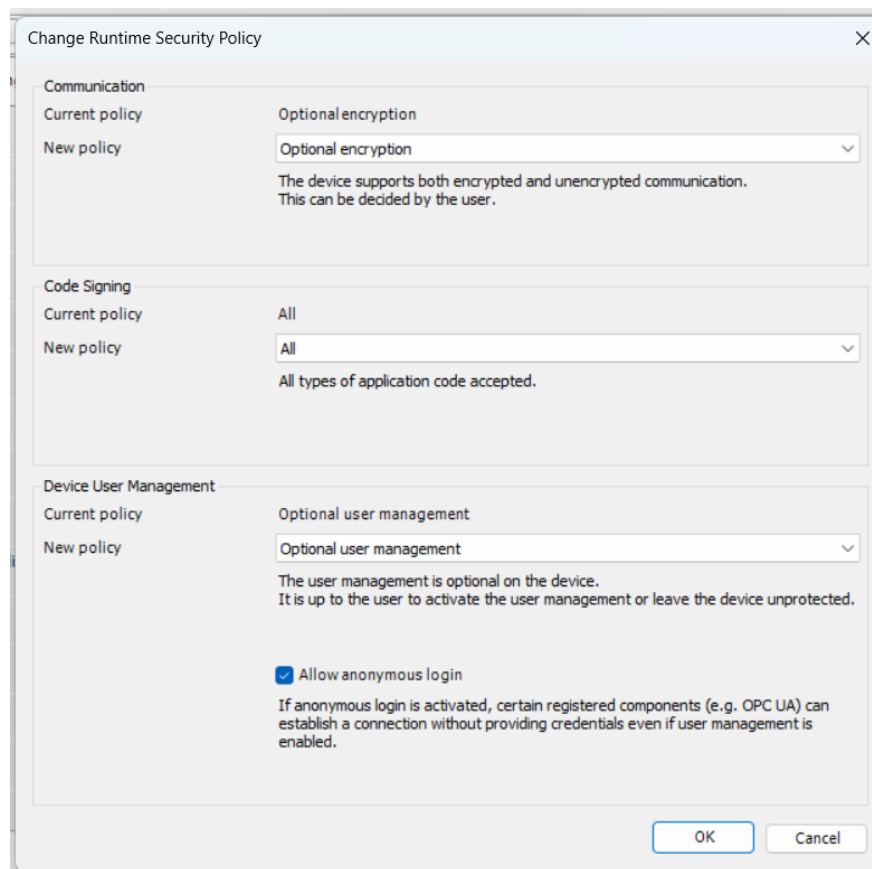


Figure 46: Runtime Security Policy dialog: Optional encryption, Code Signing = All, Optional user management, anonymous login enabled.

## 0.7 First-Login Credentials

1. On first connection to the device, CODESYS prompts for a new device username and password.
2. Record these credentials immediately. There is no recovery flow for a lost device password — losing it requires reinstalling the runtime and reconfiguring the device from scratch.
3. Note the location of the **Users and Groups** panel in the device tree for later reference; this is where device credentials are managed after the initial login.

## 0.8 Simulink OPC UA Client Configuration

1. In Simulink, open **Configuration Parameters** and navigate to the **OPC Configuration** pane.
2. Add an OPC UA client pointing at the CODESYS runtime's endpoint.
3. Check the connection status indicator to confirm the client connects successfully to the running CODESYS server (Figure 47).

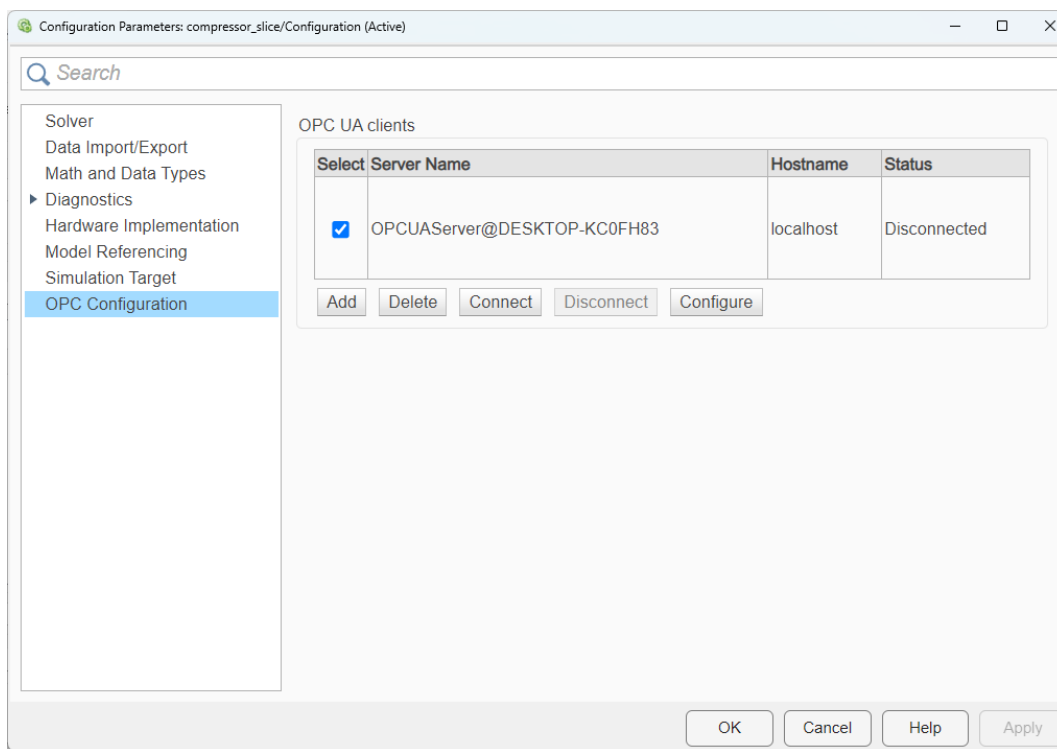


Figure 47: Simulink Configuration Parameters, OPC Configuration pane, showing the configured client and its connection status.

## 0.9 Re-Browsing OPC UA Nodes After a CODESYS Rebuild

1. After any CODESYS project rebuild followed by a download to the runtime, return to every OPC UA Read and OPC UA Write block in the Simulink model.
2. Re-browse the OPC UA node tree in each block (Figure 48) and re-select the target node, even if the node's name and path appear unchanged.
3. This step exists because a Download event regenerates the OPC UA address space on the CODESYS side; see the Known Gotchas subsection below for the underlying reason.

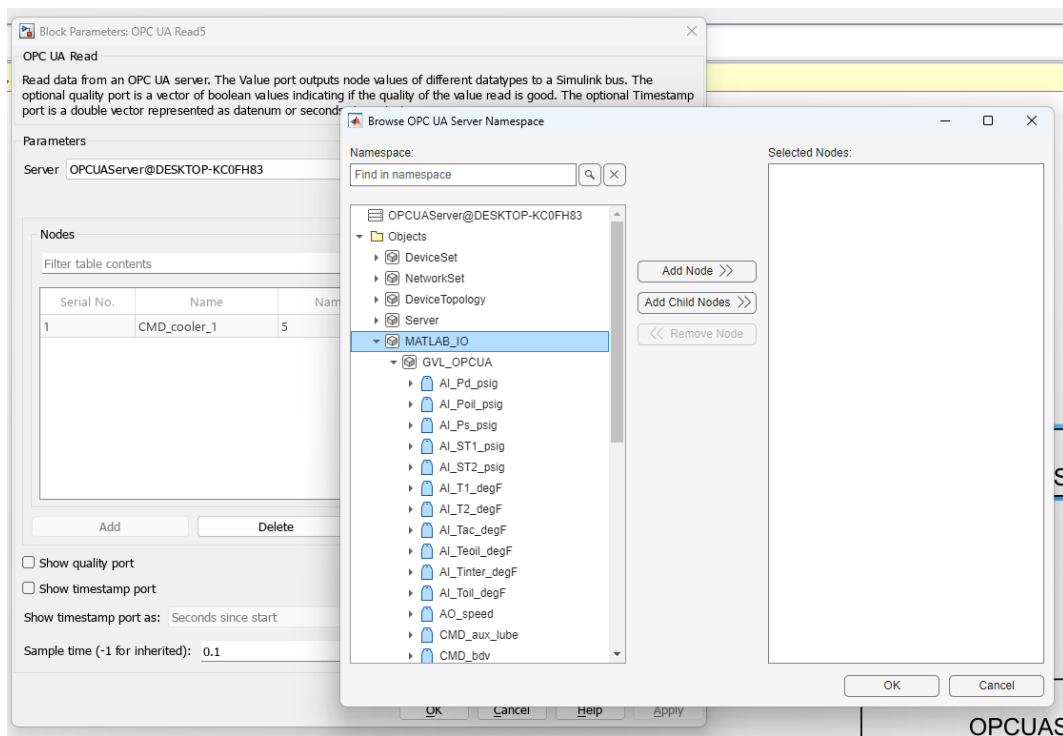


Figure 48: OPC UA namespace browse dialog used to re-select a node inside a Simulink OPC UA Read/Write block after a CODESYS rebuild.

## 0.10 Known Gotchas

- **Demo mode timeout.** CODESYS demo mode stops the runtime after approximately two hours of continuous operation. This does not require a reinstall — relaunch the CODESYS Control Win service and reconnect.
- **Address space invalidation on Download.** Any Download event in CODESYS regenerates the OPC UA address space, which invalidates every cached node reference held by Simulink. After any CODESYS rebuild and download, every OPC UA Read and Write block in Simulink must be re-browsed and refreshed (Section 0.9); a block left with a stale cached reference will fail silently or read/write a stale node.

- **Discovery-URL fallback.** If MATLAB cannot locate the OPC UA server directly, try connecting via the discovery URL instead of the direct endpoint URL.